

ProGenome: a federated workflow for genome-graph and proteomic integration

Complete knowledge-transfer document: problem, biology, data, pipeline, knowledge graph, models, federated learning, deployment and results

Team #3, Nordic Biobank x NVIDIA Federated Learning Hackathon, Copenhagen, September 2026. E

18 September 2026

Contents

1	How to read this document	3
2	Part I: Primer	3
2.1	Biology in ten minutes	3
2.2	Software in ten minutes	3
2.3	Data science in ten minutes	3
2.4	Machine learning in ten minutes	4
2.5	Federated learning in five minutes	4
3	Part II: The workflow on a few pages	4
3.1	The story, stage by stage	4
3.2	Where each data source is used, and where it is not	5
3.3	What the trained model gives you, and what is done with it	6
3.4	Ground truth versus prediction	7
3.5	Where the language model sits, and what it achieves	9
3.6	Connecting the dots to the README on branch main	10
4	Part III: The project	12
4.1	1. Problem statement and goal	12
4.2	2. Data sources: what, where, why	12
4.3	3. The data pipeline	13
4.4	4. Exploratory data analysis	16
4.5	5. Statistics: does the graph co-occur with phenotypes?	19
4.6	6. The knowledge graph	19
4.7	7. From graph to embeddings	22
4.8	8. The encoder: a heterogeneous graph neural network	25
4.8.1	Metrics, defined	27
4.9	9. Integrating the proteome (schema v2)	28

4.10	10. The decoder: GraphRAG with an NVIDIA NIM language model	28
4.11	11. Federated learning with NVFlare	30
4.11.1	With and without federation, on this data	32
4.11.2	Why federated, and what kind	33
4.11.3	Mechanics in detail	33
4.12	12. System design, tech stack and deployment	34
4.12.1	The Docker image, layer by layer	35
4.12.2	GPU deployment on Brev, step by step	36
4.12.3	Inference and what torch.compile does	36
4.13	13. How the goals were met, and what is not claimed	36
4.13.1	Verification and reproduction (18 September, before the commit)	37
4.14	14. Team context	38
4.14.1	What is on main since this branch was created, and how it relates	38
5	Part IV: Reference	38
5.1	Run it yourself	38
5.2	Secrets and configuration	39
5.3	Repository map (genomics/)	39
5.4	Glossary	39

1 How to read this document

This is written for a teammate who joins today and knows nothing about the biology, the software, the models or the infrastructure. Part I is a primer that defines every concept used later. Part II is the whole workflow on a few pages: what goes in and what comes out of every stage, where each data source is used, what the trained model produces and how it is used, ground truth against prediction, where the language model sits, and how every item of the team README on branch main is covered. Part III is the project in depth, in the order the data flows: problem, data sources, data pipeline, exploratory analysis, knowledge graph, embeddings, the graph neural network (the encoder), the language-model decoder, the federated training, deployment, results and caveats. Part IV is how to run everything, a glossary and a repository map. Every number is measured on chromosome 22 with random seed 42 and comes from files under genomics/outputs; nothing is typed in from memory.

2 Part I: Primer

2.1 Biology in ten minutes

DNA is a long text written in four letters (A, C, G, T). Humans have about 3 billion letters, packaged in 23 pairs of chromosomes; chromosome 22 is one of the smallest, about 50 million letters. Everyone carries two copies of each chromosome, one from each parent. A gene is a region of DNA that encodes a protein; proteins are the molecules that do the work in cells, and measuring how much of each protein a person has is called proteomics.

Two people's DNA differs at roughly one letter in a thousand; those positions are variants. A haplotype is the specific combination of variants along one copy of a chromosome. Because DNA is inherited in chunks (recombination cuts and rejoins the parental copies at a limited number of places), neighbouring variants tend to travel together. A haploblock is a stretch of chromosome between recombination hotspots that is usually inherited as one unit. Phased data means we know, for every variant, which of the two copies it sits on, so each person's two haplotypes per block are known separately.

The 1000 Genomes Project (1000G) sequenced 2,548 people from 26 populations grouped into five continental ancestries: AFR (African), AMR (admixed American), EAS (East Asian), EUR (European) and SAS (South Asian). It recorded only ancestry, population and sex about them, nothing clinical. A phenotype is any observable property of a person; in this project the real phenotypes are those three, and a clinical-looking case/control phenotype had to be simulated.

2.2 Software in ten minutes

Python is the language of every script here; pandas handles tables, scipy handles sparse matrices (mostly-zero grids stored compactly), PyTorch does neural networks and PyTorch Geometric (PyG) adds graph neural networks. Git tracks versions of the code; a branch is a parallel line of work (ours is called modelling). Docker packages the code and every library it needs into an image so it runs identically on any machine. A GPU is a processor built for many small parallel calculations; CUDA is NVIDIA's software that lets PyTorch use it. NVIDIA Brev rents GPU machines by the hour. Neo4j is a database made for graphs, with a browser to look at them.

2.3 Data science in ten minutes

Exploratory data analysis (EDA) means measuring and plotting the data before modelling: sizes, distributions, missing values, obvious structure. To judge a model honestly the people are split once into train (learn), validation (choose settings and when to stop) and test (report only once, at

the end); here 70/15/15 percent, chosen at random but stratified so each ancestry is represented in every part. Accuracy is the fraction correct; balanced accuracy averages the accuracy per class so a rare class cannot be ignored; AUC (area under the ROC curve) is the probability that a random case is scored higher than a random control, 0.5 is guessing and 1.0 is perfect. A negative control is a target that must come out at chance level if the method is sound.

2.4 Machine learning in ten minutes

An embedding is a list of numbers (a vector) that represents an object so that similar objects get similar vectors. A graph is a set of nodes joined by edges; a knowledge graph is a graph whose nodes and edges have types and properties. A graph neural network (GNN) computes a vector for every node by repeatedly mixing each node’s own vector with those of its neighbours (message passing); after two rounds a node’s vector summarises its two-hop neighbourhood. Training means adjusting the network’s weights so that a prediction made from those vectors matches known labels, measured by a loss; an optimiser (Adam) nudges the weights to reduce the loss, one epoch being one pass over the data. An encoder turns raw data into vectors; a decoder turns vectors (and retrieved facts) into an output, here text written by a large language model (LLM). Retrieval-augmented generation (RAG) gives the LLM the facts it should use so it does not invent them; GraphRAG retrieves those facts from a graph.

2.5 Federated learning in five minutes

Hospitals cannot pool patient data. Federated learning trains one model across sites without moving records: each site trains the shared model on its own data for a few steps, sends only the updated weights to a coordinator, the coordinator averages them (FedAvg) and sends the average back; repeat for several rounds. NVIDIA FLARE (NVFlare) is the framework that runs this loop; its simulator runs all sites on one machine so the workflow can be tested before real deployment.

3 Part II: The workflow on a few pages

3.1 The story, stage by stage

Read the workflow as one sentence first: a public graph of haplotype clusters is joined to people by their sample id; each person’s row of that graph, their phenotype labels and their protein measurements are attached to a person node; a graph neural network is trained to predict a label from the person’s neighbourhood; the trained network yields predictions, an embedding per person and cluster, a saliency per cluster and a set of weights; the predictions and embeddings are checked against ground truth, a language model turns them into a cited report per person, and the weights are what a federated deployment exchanges instead of data. Every stage below is one script with one Makefile target.

Stage	What goes in	What comes out	Lands in
1 download	URLs data.haploblocks.org	on HaploGraph node and edge files, phenotypes, block statistics, md5-verified	data/
2 knowledge graph v1	nodes.csv.gz (who carries which cluster), edges (which clusters co-occur), block statistics, phenotypes	Individual, Cluster, Block nodes; CARRIES, CO_OCCURS, IN_BLOCK, NEXT_BLOCK edges; labels stored on the person nodes	outputs/kg/chr22/(carries.npz, hetero.pt)

Stage	What goes in	What comes out	Lands in
3 statistics	the graph and the labels	for every cluster and edge, how strongly it tracks ancestry, population and sex (Cramer’s V, FDR)	outputs/cooccurrence/chr22
4 baseline and split	carrier matrix and labels	logistic-regression scores to beat; the single train/val/test split every later model reuses	outputs/baseline/, outputs/splits/
5 starting embeddings	carrier matrix only, no labels	a 32-number vector per person and per cluster (SVD), or Node2Vec, learned, or the raw row	computed inside training
6 GNN v1 (encoder)	graph, starting vectors, and the labels of training people only	trained weights; class probabilities per person; a 64-number embedding per person and per cluster	outputs/gnn/chr22/<target>
7 embedding check	SVD and GNN embeddings, labels of test people	silhouette and nearest-neighbour accuracy, 2-D plots	outputs/embeddings/chr22
v2.1 synthetic proteomics	the real 1000G ids, the carrier matrix, the gene BED	3 per-site protein matrices, metadata (site, age, sex, case/control), ground_truth.json	outputs/proteomics_synthetic
v2.2 knowledge graph v2	graph v1, gene BED, proteomics	Gene and Protein nodes; OVERLAPS, ENCODES, MEASURED edges (harmonised z); site and phenotype labels on people	outputs/kg/chr22/hetero_
v2.3 EDA	everything above	the eight-section report with tables and plots	outputs/eda/chr22/EDA.m
v2.4 GNN v2	graph v2; modality genome / proteome / both; target phenotype / site / ancestry / sex / proteome	metrics, test predictions, embeddings, saliency per cluster, training history	outputs/gnn_v2/chr22/<target>
v2.5 ridge	carrier matrix, protein z-scores	per-protein test R-squared, split into cis and other proteins	outputs/gnn_v2/chr22/protein
v2.6 decoder	one person: graph neighbourhood, GNN prediction and embedding neighbours, saliency	a cited JSON insight written by the NIM language model, with a citation check	outputs/graphrag/chr22/<person>
v2.7 federated	graph v2 split by site, the model definition	a global model trained without moving any person’s data; its score on the same held-out people	outputs/federated/chr22/
inference	a trained run	predictions and embeddings for all 2,548 people; eager versus compiled timing	outputs/gnn/chr22/<run>

3.2 Where each data source is used, and where it is not

The most common confusion is what the phenotypes and the proteomics do. The phenotypes are labels: they are stored on the person node as training targets and evaluation ground truth and never become an edge, a node feature or an input to the starting embeddings. The proteomics enters three times: as MEASURED edges from a person to the proteins observed in them, as part of the person’s input vector, and, through its metadata, as two more labels (site and the synthetic case/control phenotype). The HaploGraph provides the structure everyone shares.

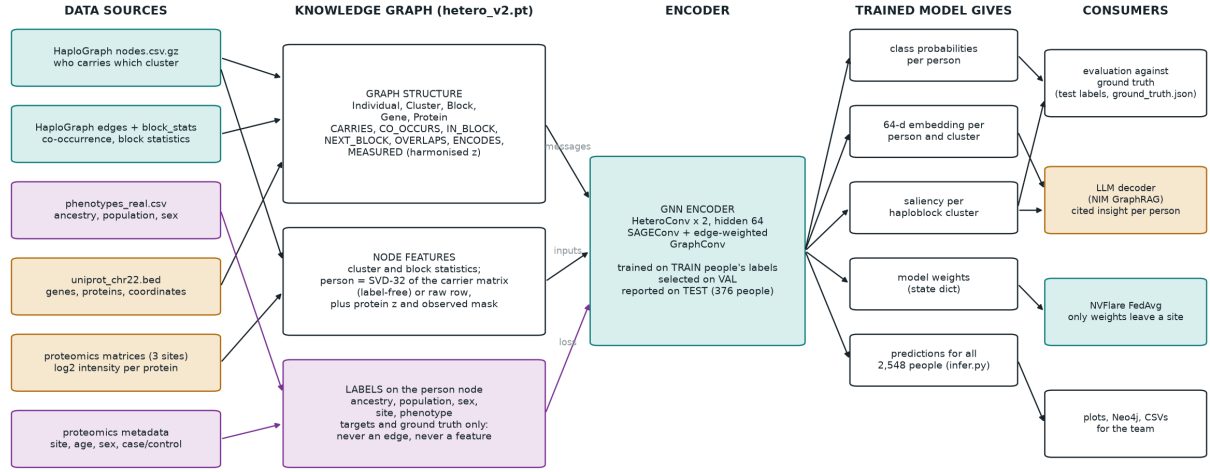


Figure 1: Where each data source enters, what the trained model produces, and where each product goes.

Source	Used for	Never used for
HaploGraph nodes.csv.gz	CARRIES edges; the carrier matrix behind the SVD starting vectors, the logistic baseline, the raw input, the saliency and the ridge	labels
HaploGraph edges_lift_above_threshold.csv.gz	CO_OCCURS edges with weight = normalised log lift	anything about people directly
block_stats.tsv, boundaries	Block node features; five of the seven Cluster features; NEXT_BLOCK order; block-gene overlaps	labels
phenotypes_real.csv (ancestry, population, sex)	labels on Individual nodes: the training target for training people, the ground truth for validation and test people, the classes in the Cramer's V tests, the colours in plots	edges, node features, the SVD (which is label-free), the test split (only its stratification)
uniprot_chr22.bed	Gene and Protein nodes with span and isoform count; OVERLAPS by coordinate; ENCODES	labels
proteomics matrices (per site, log2)	harmonised per site and protein into z; MEASURED edge weights; the person's input vector (z plus observed mask); the proteome-only MLP	cross-site normalisation (the harmoniser never sees two sites at once)
proteomics metadata (site, age, sex, phenotype)	site = batch control and the federated partition; phenotype = the case/control target; age = decoder context	edges or node features
ground_truth.json	scoring only: which clusters are causal (saliency precision), which proteins are cis-affected (ridge R-squared)	any model input

3.3 What the trained model gives you, and what is done with it

A trained run is a folder with model.pt (the weights), metrics.json, history.csv, test_predictions.csv, embedding_individual.npy (and embedding_cluster.npy in v1), and for the raw-input phenotype run saliency_top100.csv. Five products come out of the encoder and each has a consumer:

- Class probabilities per person: for every one of the 2,548 people, a probability per class (five

ancestries, 26 populations, case/control). Consumed by the metrics against held-out labels, by infer.py which writes predictions_all_individuals.csv with a confidence per person, and by the decoder, which quotes the prediction for the person it describes.

- A 64-number embedding per person and per cluster: the encoder’s last hidden layer. Consumed by embeddings.py (silhouette, nearest-neighbour accuracy, PCA plots), by the decoder (the five nearest people in this space are part of the retrieved context) and available for any downstream clustering of people into phenotype groups.
- A saliency per haploblock cluster: the gradient of the case score with respect to the person’s carrier row, averaged over test cases. Consumed by the ground-truth check (how many of the top 20 are planted causal clusters) and by the decoder, which lists the globally salient clusters the person carries.
- The weights themselves: in the federated setting the weights are the only thing that leaves a site, so the same model definition (federated/model.py) is what NVFlare averages.
- A timed inference path: infer.py rebuilds the run’s inputs, scores the whole graph in one pass (36 ms eager, 4.9 ms with torch.compile on the A100) and writes predictions and embeddings for everyone.

3.4 Ground truth versus prediction

Every model is scored on the same 376 held-out people that no model saw during training or model selection. For the real labels the ground truth is the 1000G panel; for the synthetic phenotype the ground truth is the label the generator drew from the planted causal clusters; for the saliency and the ridge it is the list of causal clusters and cis proteins in ground_truth.json; for the federated model it is the same 376 people scored centrally. The confusion tables below are read row = truth, column = prediction.

Ancestry, true / predicted	AFR	AMR	EAS	EUR	SAS
AFR (99)	99	0	0	0	0
AMR (52)	1	48	0	3	0
EAS (76)	0	0	76	0	0
EUR (76)	0	4	0	72	0
SAS (73)	0	0	0	0	73

368 of 376 correct (balanced accuracy 0.974). The eight errors are all between AMR and EUR, which is what admixture predicts: the AMR panel populations carry European haplotypes.

Synthetic phenotype, true / predicted	case	control	correct
genome plus proteome (graph): case (136)	128	8	367 / 376, AUC 0.992
genome plus proteome (graph): control (240)	1	239	
proteome only (MLP): case (136)	129	7	364 / 376, AUC 0.962
proteome only (MLP): control (240)	5	235	
genome only (graph): case (136)	95	41	202 / 376, AUC 0.601
genome only (graph): control (240)	133	107	
sex, negative control: female (188)	119	69	189 / 376, chance
sex, negative control: male (188)	118	70	

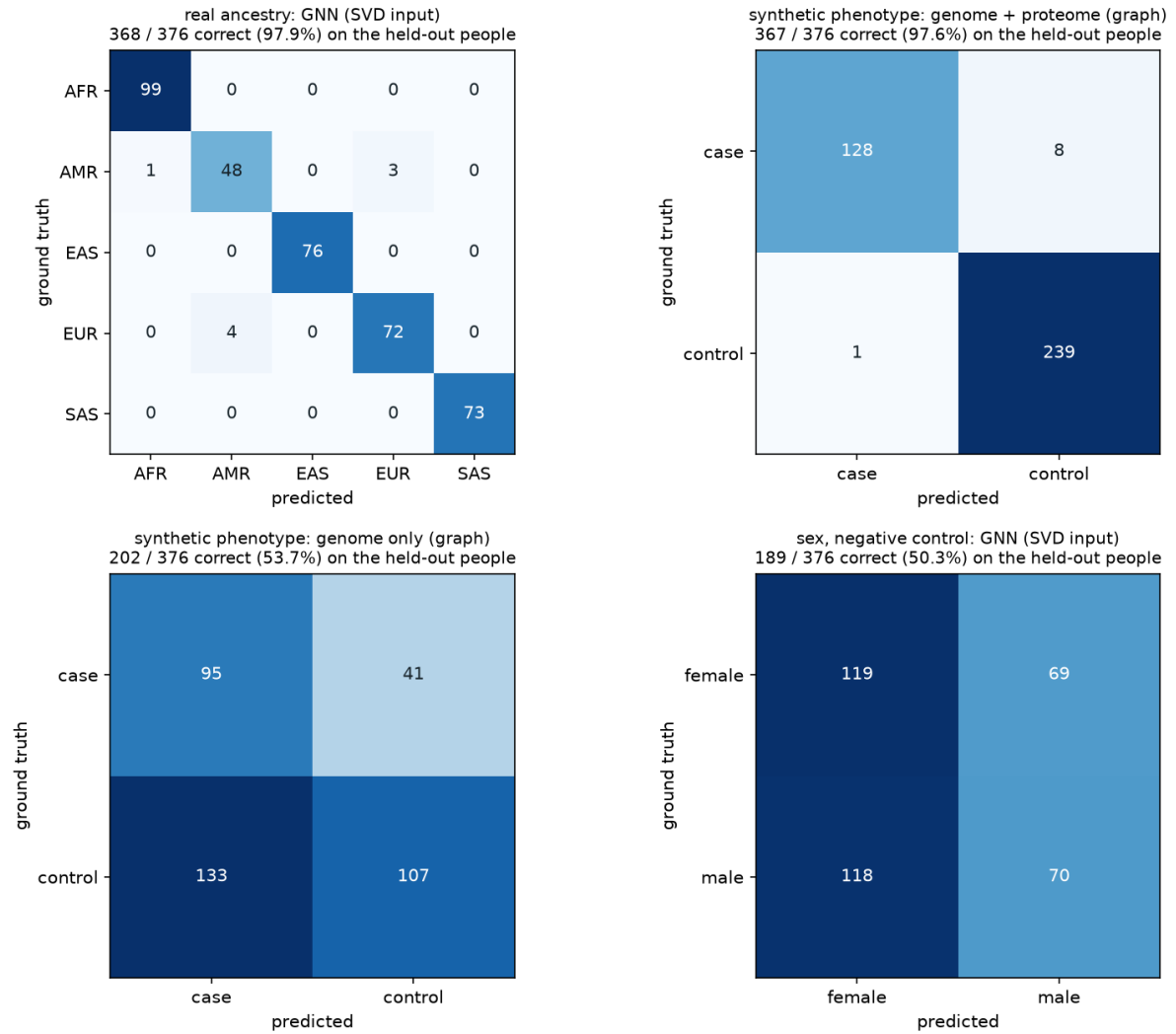


Figure 2: Ground truth versus prediction on the 376 held-out people: real ancestry (GNN, SVD input), the synthetic phenotype from genome plus proteome, from the genome alone, and sex, the negative control.

The genome alone gets little more than half right because the planted genomic signal is 20 clusters through a noisy logistic link; the proteome carries most of the signal; the graph that joins the two makes the fewest errors, with a single false case. Sex on an autosome is a coin flip, as it must be.

Rank	Cluster (saliency of the combined model)	Saliency	Planted causal cluster?
1	chr22_46902935-46974137_cluster186	0.110	yes
2	chr22_44263132-44288496_cluster19	0.080	yes
3	chr22_25066667-25206817_cluster346	0.077	yes
4	chr22_46974137-47051453_cluster43	0.055	no
5	chr22_44166469-44185056_cluster231	0.054	yes
6	chr22_47257518-47285160_cluster1	0.046	no
7	chr22_40032702-40132216_cluster25	0.046	no
8	chr22_49408121-49430759_cluster3	0.043	no
9-20	twelve further clusters	0.042-0.036	no

Four of the top twenty (and the top three outright) are among the 20 planted causal clusters out of 6,551; by chance 0.06 would be. For the genome-to-protein direction the ridge finds 4 of the 20 cis proteins with test R-squared above 0.1 and none of the 440 others. The federated global model, scored on the same 376 people, reaches AUC 0.998 and balanced accuracy 0.963 against the central model’s 0.992 and 0.969.

3.5 Where the language model sits, and what it achieves

The language model is not part of training and makes no prediction. It sits after the encoder, once per person, as the decoder: `graphrag_decoder.py` walks the graph around one person and collects facts (their ancestry-informative clusters with blocks, genes and proteins; their most extreme protein levels and the block that encodes each; the GNN’s prediction; the salient clusters they carry; their five nearest neighbours in the embedding), serialises them as JSON and asks the NIM model (nvidia/nemotron-3-super-120b-a12b) to write a structured report using only those facts and citing every id verbatim. The code then checks each cited id against the context. What this achieves is the last step the README’s mission implies, turning numbers into an insight a research team can read: a summary, an ancestry assessment, a phenotype assessment, explicit genome-to-proteome links and caveats, each traceable to graph ids. For HG00103 (EUR, GBR, 994 clusters carried, predicted control) the model answered in 13 seconds from a 4,555-token context with a 1,020-token reply, cited 25 ids and invented none; its links named, for example, cluster `chr22_40032702-40132216_cluster151` in the block encoding TNRC6B (protein Q9UPQ9) and cluster `chr22_26024448-26060666_cluster35` in the block encoding MYO18B (Q8IUG5), and its caveats stated that the phenotype is synthetic and that ancestry is population structure, not a medical finding. Without an API key the same script prints the exact prompt (dry run), so the retrieval can be inspected offline.

Retrieved fact for HG00103	Value
GNN prediction	control (truth: control)
globally salient clusters carried	chr22_46902935-46974137_cluster186 (0.110), chr22_46974137-47051453_cluster43 (0.055), chr22_40032702-40132216_cluster25 (0.046)
most ancestry-informative clusters, with block genes and proteins	cluster151 of block 40032702-40132216 (TNRC6B, Q9UPQ9); cluster545 of 35738379-36060254 (RBFOX2, O43251); cluster35 of 26024448-26060666 (MYO18B, Q8IUG5)

Retrieved fact for HG00103	Value
most extreme protein levels (z)	H1-0 +2.75, RIMBP3 -2.65, INPP5J +2.53, CRYBA4 +2.52, YWHAH +2.37
nearest people in the embedding	HG01767 (EUR), NA12842 (EUR), HG00351 (EUR), HG04002 (SAS), HG00336 (EUR)

Decoder field	Text returned (Nemotron 3 Super, 13.3 s, 25 ids cited, 0 unknown)
summary	Individual HG00103 is of EUR ancestry (GBR population) and carries 994 haploblock clusters. The GNN phenotype prediction for this individual is 'control', which is a synthetic case/control label used for pipeline testing. Several ancestry-informative clusters are carried, showing high enrichment in various populations including EUR, EAS, and SAS.
ancestry_assessment	The individual's ancestry is EUR (European) as per the 1000G panel data, with population label GBR. This reflects population structure, not a medical finding.
phenotype_assessment	The GNN-predicted phenotype is 'control', which is a synthetic case/control label used to test the pipeline; it does not reflect a real medical condition.
genome_proteome_links	cluster151 → block 40032702-40132216 → TNRC6B → Q9UPQ9 (carried, enriched in EAS); cluster545 → RBFOX2 → O43251; cluster35 → MYO18B → Q8IUG5 (enriched in EUR)
caveats	ancestry is population structure, not a medical finding; the phenotype is synthetic; not all clusters have gene or protein annotations; the prediction comes from the embedding and may not match observed protein levels

What the LLM adds is the sentence layer: a readable paragraph, the genome-to-protein chain spelled out per person, the right hedges attached automatically, graceful handling of missing information (for HG00096, who is not in the held-out split, it reported that no prediction exists instead of inventing one), and traceability, because every claim carries an id that the code verifies. It makes no prediction and adds no outside knowledge.

3.6 Connecting the dots to the README on branch main

The team README fixes a mission, three research questions, a chromosome-22 demo scope, five required datasets and a data-integration flowchart. Each maps to something concrete in genomics/:

README item	What it became	Evidence
Mission: each institution keeps individual-level data locally, trains the same graph model, exchanges only model updates; a server aggregates and returns them	the person is its own node type, so a site holds only its people and their CAR-RIES and MEASURED edges while the cluster/block/gene/protein graph is public; NVFlare FedAvg with client.py sending a state dict only	outputs/federated/chr22/evaluation AUC 0.998 vs central 0.992

README item	What it became	Evidence
Opening line: a variant-based phenotype-propensity reference graph combined with patient-specific proteomics, what can we learn?	reference graph = the shared cluster/block/gene/protein layer; patient-specific = Individual nodes with CARRIES and MEASURED edges; learned: integration beats either modality alone, cis effects need sparse models, federation costs nothing when sites are alike	sections 9, 11, 13
Background: one gene gives many protein products	917 UniProt isoform rows collapsed to 460 proteins with an isoform count as a Protein feature; ENCODES keeps gene to protein explicit	haplogk_proteins.load_protein_bec
RQ1: connect haploblock genomics to genes and proteomic data in a graph model	schema v2, five node types, seven edge types, one join key; the README flowchart maps one-to-one: Participant = Individual, Haploblock hash = Cluster, Haploblock = Block, Encoded protein = Gene ENCODES Protein, Measured abundance = MEASURED edge	section 6; Neo4j browser
RQ2: can a GNN combine genomic and proteomic information to identify disease-related phenotype clusters	yes, in both senses of cluster: groups of people (AUC 0.60 genome, 0.96 proteome, 0.99 both; embedding silhouette 0.70) and haploblock clusters tied to the phenotype (saliency top 3 all causal); the person-level graph, with every participant as a node, is what lets the GNN use both modalities	sections 8, 9; Part II ground truth tables
RQ3 (aspirational): train across institutions without transferring individual-level data	done in NVFlare simulation with three sites; real-machine POC is the next step	section 11
Demo scope: chromosome 22 first	everything runs on chr22; nothing is chromosome-specific (CHROM=chr21 make run)	Makefile
Dataset 1: haploblock BED and per-individual haploblock hashes	boundaries and block_stats from haploblocks.org; the hashes were already clustered upstream into the HaploGraph clusters we consume, so no hash was recomputed	fetch_data.sh
Dataset 3: gene BED for chr22	uniprot_chr22.bed (team); Block OVERLAPS Gene by coordinate intersection, 1,063 edges, 29 genes outside every block	build_kg_v2.py
Dataset 4: gene-to-protein mapping	the same BED: UniProt accession per gene, isoforms collapsed; 460 ENCODES edges	build_kg_v2.py
Dataset 5: proteomic data for chr22 proteins	the team's 120-patient synthetic set validated the proteomics plumbing but cannot join the genome (its ids are not 1000G ids and its phenotype has no genomic cause); the joinable set on 2,503 real 1000G ids with ground truth replaces it for integration; real data (Wu 2013, UKB-PPP) is the planned next source	proteomics_synth_1000g.py

README item	What it became	Evidence
Data-integration flowchart	implemented as a PyTorch Geometric HeteroData object plus a Neo4j load, with two additions the README did not list: CO_OCCURS between clusters and NEXT_BLOCK between blocks	hetero_v2.pt; neo4j_load.py

4 Part III: The project

4.1 1. Problem statement and goal

The team's README (branch main) states the mission: develop a proof-of-concept workflow that integrates a known genome graph with proteomics, with a federated approach where each participating institution retains its individual-level data locally and trains the same graph-based model, exchanging only model updates with a coordinating server. Three research questions make it concrete:

- RQ1: How can haploblock-based genomic information be connected to genes and proteomic data in a graph-based data model?
- RQ2: Can a graph neural network combine genomic and proteomic information to identify disease-related phenotype clusters?
- RQ3 (aspirational): Can a graph neural network trained across multiple institutions predict clinical outcomes without transferring individual-level data?

Translated into engineering: build a typed graph in which a person links to the haplotype clusters they carry, clusters sit in blocks, blocks overlap genes, genes encode proteins, and the same person links to their measured protein levels (RQ1); train a GNN on it and show that genome plus proteome predicts a phenotype better than either alone while negative controls stay at chance (RQ2); split the people by site, share only the reference graph and the model weights, and show the federated model matches the central one (RQ3). The demo scope fixed by the team is chromosome 22.

What was already available: the mentor (Ben Busby) pointed the team to haploblocks.org, whose data server publishes a ready-made graph of haplotype clusters for the 1000 Genomes people, built for this hackathon at Rigshospitalet's MDxCORE unit. That is what "we don't have to build the genome graph" meant; everything downstream of it is ours.

4.2 2. Data sources: what, where, why

Data	Where it comes from	What it contains	Why we use it
HaploGraph nodes (nodes.csv.gz)	data.haploblocks.org/haplograph/1000G/chr22	248,254 haplotype clusters x 2,548 people; 1 if the person carries the cluster	The genome graph's node features; becomes our person-to-cluster edges

Data	Where it comes from	What it contains	Why we use it
HaploGraph edges (edges_lift_above_threshold.csv.gz)	same server	187,030 cluster pairs that co-occur in people more than chance ($\text{lift} \geq 5$), with weight and lift	Co-occurrence structure; the raw edges.csv is dominated by one near-universal haplotype and is not used
Block statistics (block_stats.tsv, boundaries)	same server	669 blocks on chr22: coordinates, length, number of clusters, entropy, dominance, singletons	Block nodes and their features
Phenotypes (phenotypes_real.csv)	1000G panel via IGSR, republished with the graph	ancestry, population, sex for 2,503 of the 2,548 people	The only real labels that exist; ancestry and population are targets, sex is the negative control
Protein coordinates (uniprot_chr22.bed)	UCSC UniProt track, prepared by Friederike	917 protein isoform rows $\rightarrow$ 460 proteins, 458 genes, with genomic spans	Block-to-gene overlaps and gene-to-protein mapping in one file
Synthetic proteomics, team version	proteomics/generate_synthetic_proteomics.py (Nolan; regenerated on main with 4,000 samples)	3,334 patients each (ids SITE1_PT0001...), age, sex, case/control, log2 intensities for 460 proteins	The team's proteomics analyses on main run on it; for the genome join a version keyed to 1000G ids was needed (next row)
Synthetic proteomics, joinable version	genomics/proteomics_synthetic_joinable.py (this work)	2,503 1000G ids, 3 mixed-ancestry sites, phenotype driven by 20 causal clusters, cis effects, batch shift, missingness; ground truth saved	Lets the integration be scored against a known answer
Planned real proteomics	Wu et al. 2013 (Nature): 95 HapMap LCLs with 1000G ids; UKB-PPP pQTL summary statistics on AWS Open Data	per-person protein levels; variant-to-protein effect sizes	Real join on the same ids; real cluster-to-protein propensity edges

How the genome graph was made upstream (haploblocks.org pipeline, Kubica et al. 2025): (1) recombination-rate peaks define haploblocks; (2) each person's two phased haplotype sequences are cut out per block from the 1000G VCF; (3) all haplotypes of a block are merged into one FASTA; (4) MMseqs2 clusters near-identical haplotypes, giving each haplotype a cluster id; (5) a compact hash encodes strand, chromosome, block, cluster and variants. A sixth step (haploblock-graph-builder) produced the HaploGraph: nodes are clusters, the node feature is the 0/1 vector over people, edges join clusters that co-occur in the same people. We did not rerun these steps; we verified their outputs agree (669 blocks in every file, identical per-block cluster counts in all 669, 248,254 clusters both ways) and consumed them.

4.3 3. The data pipeline

Everything is a numbered script under genomics/, driven by a Makefile so a fresh clone runs with two commands. Downloaded inputs and generated outputs are git-ignored and regenerated; secrets

1. chromosome 22 is cut into haploblocks at recombination hotspots (669 blocks tiling 17.1-50.2 Mb)



2. two phased haplotypes per person and block

HG00096	hap0: ACGTTGCA...	hap1: ACGATGCA...
HG00097	hap0: ACGATGCA...	hap1: TCGTTGCA...
NA21144	hap0: ACGATGCA...	hap1: ACGATGCA...

3. MMseqs2 groups near-identical haplotypes into clusters

- cluster 1: HG00096/hap0, HG00097/hap0, ... (1,983 carriers)
- cluster 2: HG00096/hap1, HG00097/hap0, NA21144/hap0+1, ... (1,142)
- cluster 3: HG00097/hap1, ... (559)

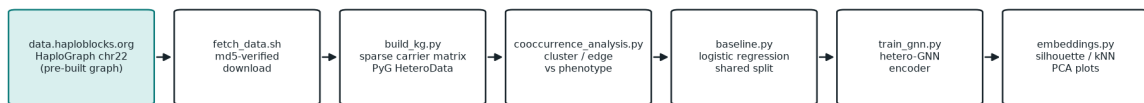
4. carrier matrix = the graph's person-to-cluster edges (1 if either haplotype is in the cluster)

	cluster 1	cluster 2	cluster 3	...
HG00096	1	1	0	...
HG00097	1	1	1	...
NA21144	0	1	0	...

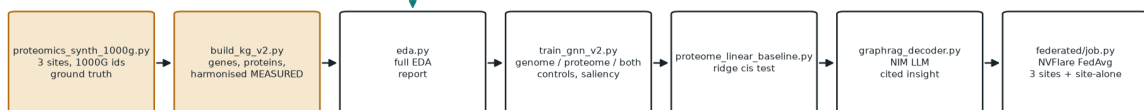
- 5. filter: keep clusters with $25 \leq \text{carriers} \leq N-25$: 248,254 -> 6,551
- 6. co-occurrence: clusters found together in people more than chance -> CO_OCCURS edges (lift)
- 7. every person carries ~928 kept clusters (2 haplotypes x 669 blocks, minus filtered ones)
- 8. the carrier matrix is the CARRIES edge list; the SVD of it gives the starting embeddings

Figure 3: From genomes to a graph: blocks, phased haplotypes, MMseqs2 clusters, the carrier matrix and the filters that produce the 6,551 kept clusters.

v1 genome graph -> phenotypes



v2 + proteomics -> integration -> decoder -> federated



Every stage is one script with a Makefile target; outputs land under genomics/outputs/<stage>/chr22/. The Docker image and brev_deploy.sh run the same chain on a GPU.

Figure 4: The two pipeline chains (v1 genome graph, v2 proteomics integration), one script per stage.

are read from environment variables and never stored in the repository.

Step	Script	What it does	Output
1	fetch_data.sh	downloads the chr22 HaploGraph files, phenotypes, block statistics; verifies md5 checksums	data/
2	build_kg.py (hap-log.py)	streams nodes.csv.gz as int8 in 8,192-row chunks into a sparse matrix; filters clusters; joins phenotypes; maps edges; writes tables and the PyG graph	outputs/kg/chr22/ (carries.npz, hetero.pt)
3	cooccurrence_analysis.py	cluster x phenotype tests, edge x phenotype similarity, per-block informativeness, plots	outputs/cooccurrence/chr22/
4	baseline.py	logistic regression on the carrier matrix; writes the shared train/val/test split	outputs/baseline/, outputs/splits/
5	graph_explore.py	NetworkX statistics, GraphML export, region and chromosome-wide plots	outputs/graph/chr22/
6	train_gnn.py	the genome-only GNN with SVD / Node2Vec / learned / raw inputs	outputs/gnn/chr22/
7	embeddings.py	quality of SVD and GNN embeddings: silhouette, nearest-neighbour accuracy, PCA plots	outputs/embeddings/chr22/
v2.1	proteomics_synth_1000g.py	synthetic proteomics on the real 1000G ids with saved ground truth	outputs/proteomics_synth/chr22/
v2.2	build_kg_v2.py (hap-logk_proteins.py)	adds genes, proteins, block-gene overlaps, harmonised measurements	outputs/kg/chr22/hetero_v2.pt
v2.3	eda.py	the full exploratory report with tables and plots	outputs/eda/chr22/EDA.md
v2.4	train_gnn_v2.py	genome / proteome / both ablations, controls, saliency, proteome regression	outputs/gnn_v2/chr22/
v2.5	proteome_linear_baseline.py	protein ridge: can the genome predict each protein?	outputs/gnn_v2/chr22/proteome/
v2.6	graphrag_decoder.py	graph retrieval + NVIDIA NIM LLM → cited insight per person	outputs/graphrag/chr22/
v2.7	federated/job.py, client.py, model.py, evaluate_global.py	NVFlare FedAvg over 3 sites and central scoring of the global model	outputs/federated/chr22/
-	infer.py	inference benchmark: eager vs torch.compile vs TensorRT	outputs/gnn/.../inference/
-	neo4j_load.py, docker-compose.yml	loads the graph into Neo4j for browsing	localhost:7474

Memory arithmetic that shaped step 2: the node file is 1.3 GB as text; as a dense 64-bit matrix it would be 5 GB, as dense int8 632 MB, as a sparse matrix with 2.8 million non-zeros about 30 MB. Streaming chunks into sparse form keeps the whole build under 1 GB and 26 seconds on a laptop.

The cluster filter: a cluster is kept if between 25 and N-25 people carry it ($N = 2,548$). A cluster carried by 2,540 people is as uninformative as one carried by 8; both have only 8 people on the informative side. This mirrors the HaploGraph’s own symmetric edge filter and reproduces its node set exactly: 248,254 clusters become 6,551 and not one of the 187,030 edges loses an endpoint. 176,903 of the dropped clusters are singletons (one haplotype).

4.4 4. Exploratory data analysis

The EDA report (outputs/eda/chr22/EDA.md, generated by eda.py) has eight sections: provenance, individuals, haploblocks, clusters, co-occurrence, genes and proteins, proteomics, genome-proteome. The key measurements:

Aspect	Measurement
Individuals	2,548 people; 2,503 labelled (AFR 660, EAS 504, EUR 503, SAS 489, AMR 347), 45 unlabelled kept as nodes; 26 populations of 61-113 people; sex balanced within each ancestry; split train 1,752 / val 375 / test 376
Blocks	669 tiling 17.1-50.2 Mb with no gaps; length median 29.7 kb (5-95%: 8.9-151 kb, longest 775 kb); clusters per block median 214, max 2,806; singleton rate median 0.62; entropy median 3.15; longer blocks hold more clusters (Spearman 0.37)
Clusters	6,551 kept of 248,254; each person carries ~928 (two haplotypes x 669 blocks minus filtered); the person x cluster matrix is 14% dense
Co-occurrence edges	187,030 with lift ≥ 5 (median 5.5, max 76); only 121 join clusters of the same block; median distance between endpoints tens of Mb; 4,344 clusters have at least one edge; degree up to 358; the top hubs are all AFR-enriched rare clusters (the mega-hub artefact)
Genes / proteins	458 genes, 460 proteins, 1,063 block-gene overlaps; 29 genes outside every block (chromosome ends and gaps); 117 genes span two blocks; up to 14 genes in one block
Proteomics (synthetic)	2,503 people x 460 proteins, 3 sites of ~835; log2 range 4.9-17.0; 7.4% missing overall, up to 16% for the least abundant proteins (missing-not-at-random at the detection limit); between-site shift 0.48 log2 before harmonisation, 0.00 after; 227 proteins keep a phenotype association at FDR 5%
Genome-proteome	for the 20 ground-truth causal clusters, the correlation between carrying the cluster and its cis protein gives r-squared up to 0.54, mean 0.17, 10 of 20 above 0.1 - the ceiling any genome-to-proteome model can reach on this data

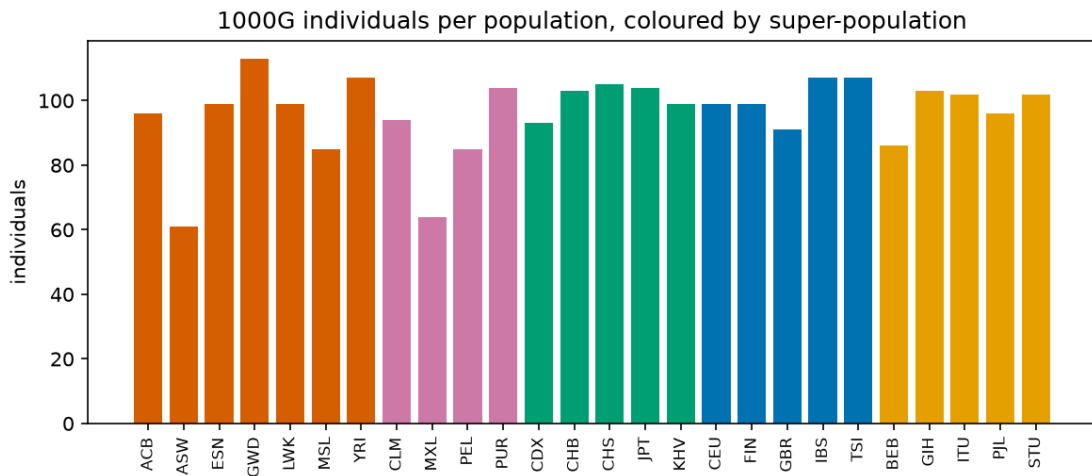


Figure 5: 1000G individuals per population, coloured by continental ancestry.

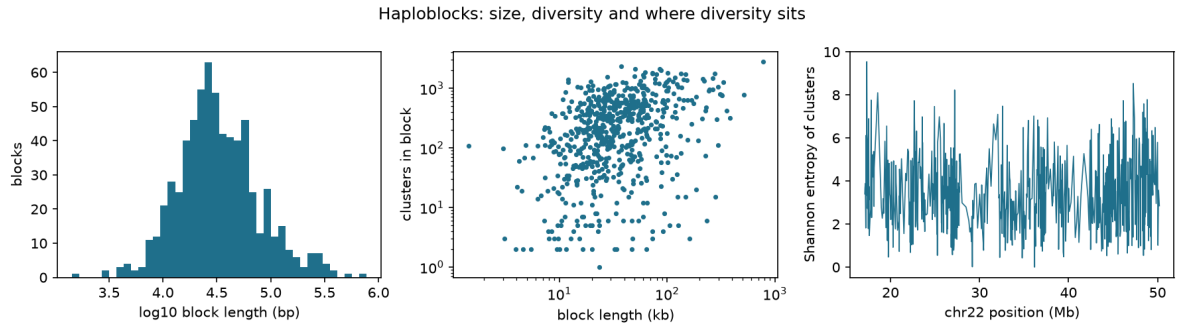


Figure 6: Haploblocks on chr22: length distribution, clusters per block versus length, Shannon entropy of clusters along the chromosome.

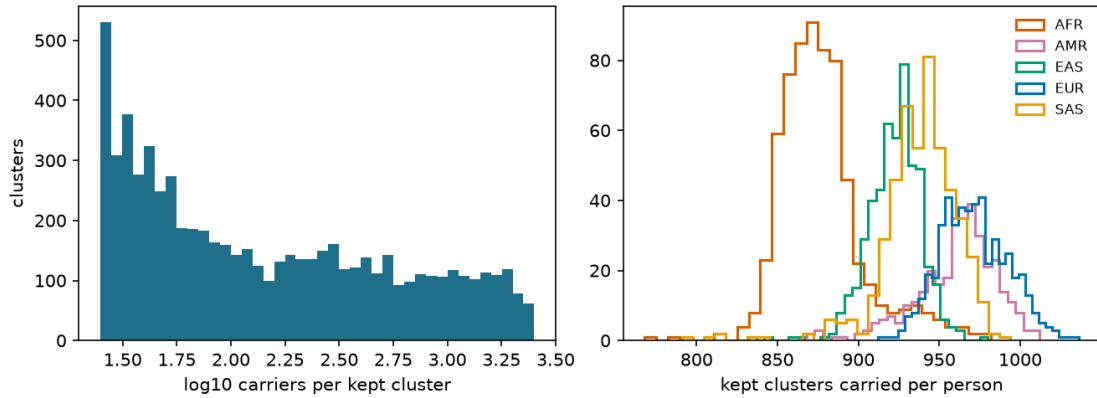


Figure 7: Kept clusters: carriers per cluster (log scale) and clusters carried per person by ancestry.

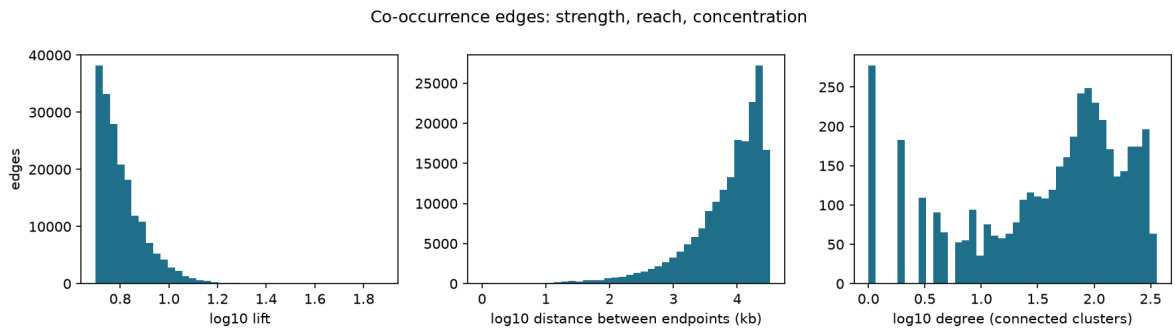


Figure 8: Co-occurrence edges: lift, distance between endpoints, degree.

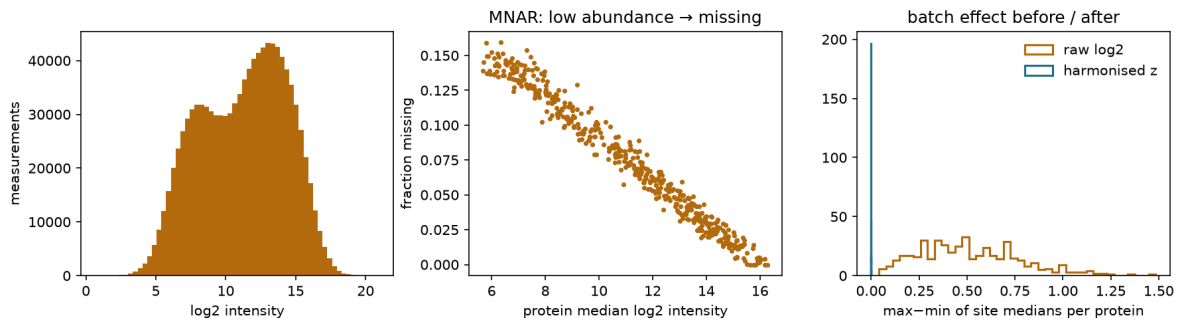


Figure 9: Synthetic proteomics: intensity distribution, missingness rising for low-abundance proteins, batch effect between sites before and after the harmoniser.

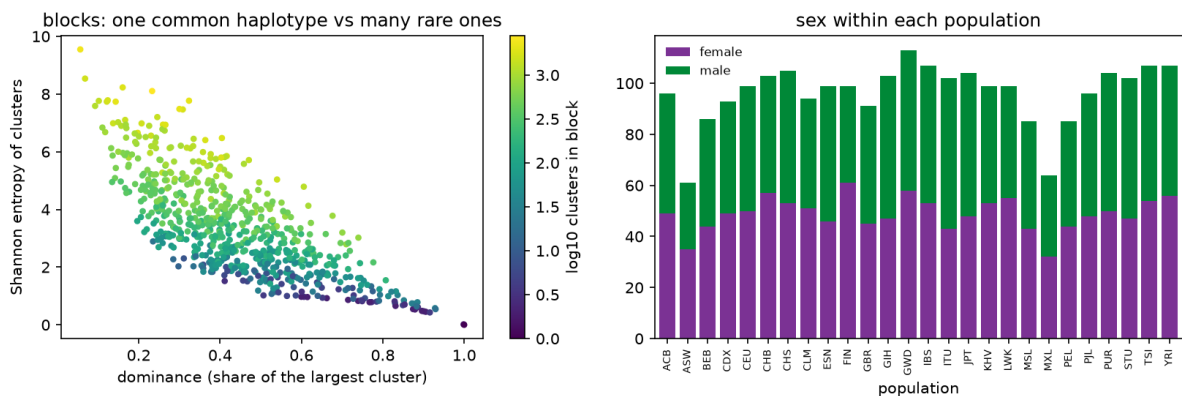


Figure 10: Blocks with one dominant haplotype versus many rare ones (left); sex within each of the 26 populations (right).

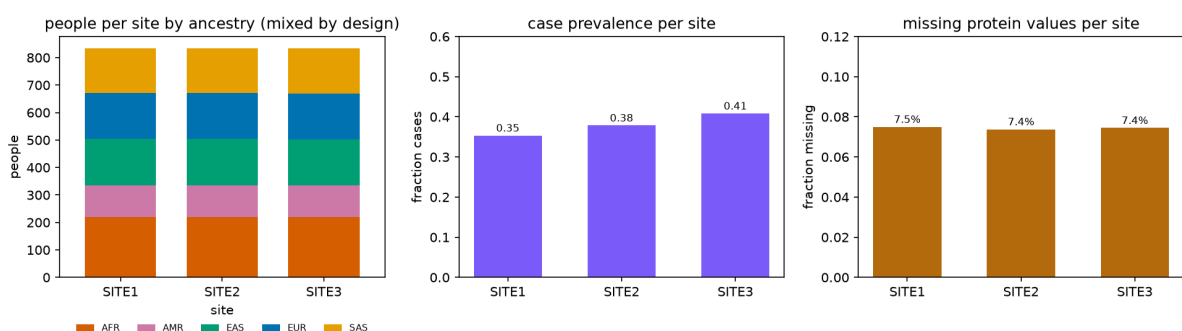


Figure 11: The three federated sites: people per site by ancestry (mixed by design), case prevalence per site, missing protein values per site.

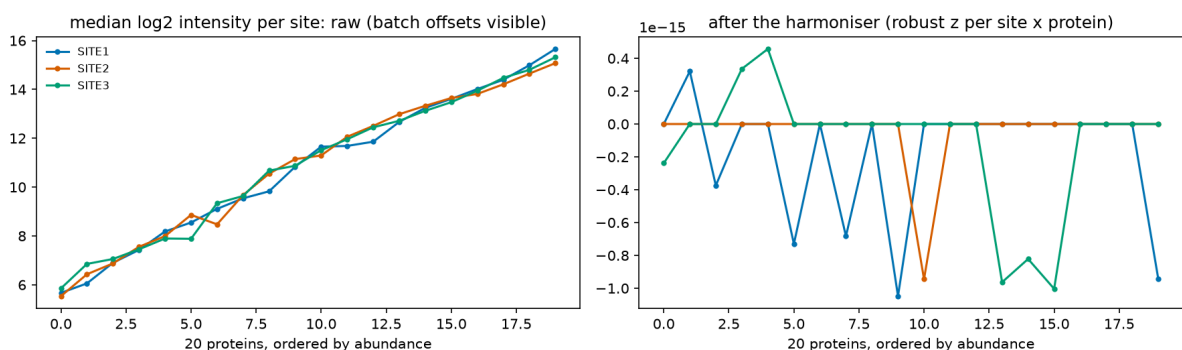


Figure 12: Per-site protein medians before and after the harmoniser: the batch offsets vanish.

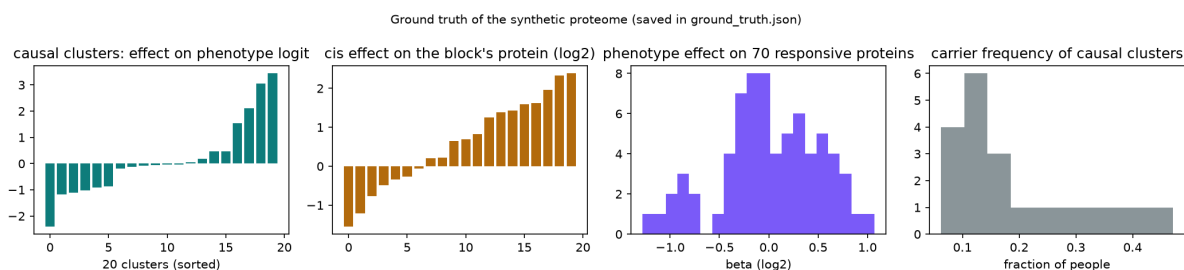


Figure 13: Ground truth of the synthetic proteome: causal-cluster effects on the phenotype, cis effects on proteins, phenotype effects on the 70 responsive proteins, carrier frequencies of the causal clusters.

4.5 5. Statistics: does the graph co-occur with phenotypes?

Before any model we asked whether the graph carries phenotype information at all. For every cluster and each label we built the $2 \times k$ table of carrier status versus class and computed a chi-square test with Cramer's V (a 0-1 effect size; for a $2 \times k$ table $V = \sqrt{\text{chi-square} / N}$), then Benjamini-Hochberg false-discovery-rate correction. All 6,551 tests run in one sparse matrix product.

- Ancestry: 6,470 of 6,551 clusters (98.8%) are associated at FDR 5%; median V 0.20, maximum 0.81.
- Population: 6,378 clusters (97.4%).
- Sex: 0 clusters; median V 0.013, maximum 0.03. This is the negative control: chr22 is autosomal, so a method that found sex signal would be fitting noise.

For the edges we compared the two endpoint clusters' ancestry-enrichment profiles: cosine similarity 0.86 for real edges versus 0.22 for degree-preserving shuffled pairs; 86% of edges join clusters enriched in the same ancestry (42% expected), and the similarity rises with lift (0.84 in the lowest lift quartile to 0.90 in the highest). Interpretation: the co-occurrence graph is largely population structure, long-range co-inheritance within ancestries rather than physical linkage.

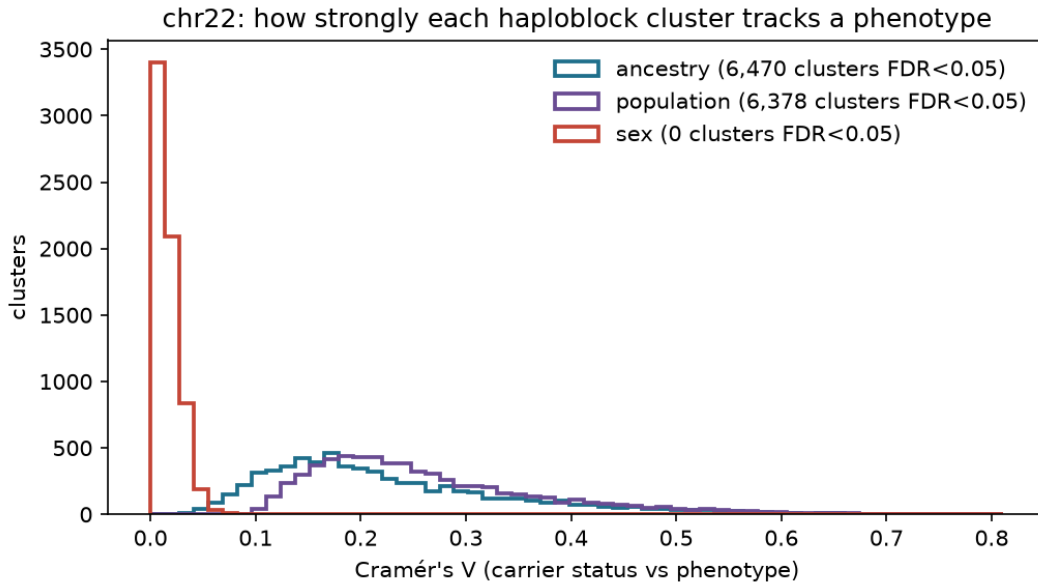


Figure 14: How strongly each cluster tracks a phenotype: ancestry and population carry signal, sex (the control) does not.

4.6 6. The knowledge graph

The HaploGraph has a single node type (cluster) and carries people only as a feature vector. We turned that vector into a second node type, the person, because a person is what phenotypes and protein measurements belong to, and what a hospital owns. The join is the 1000G sample id: the same string (for example HG00096) is a column header in nodes.csv.gz, a row key in phenotypes_real.csv and a column in the proteomics matrices.

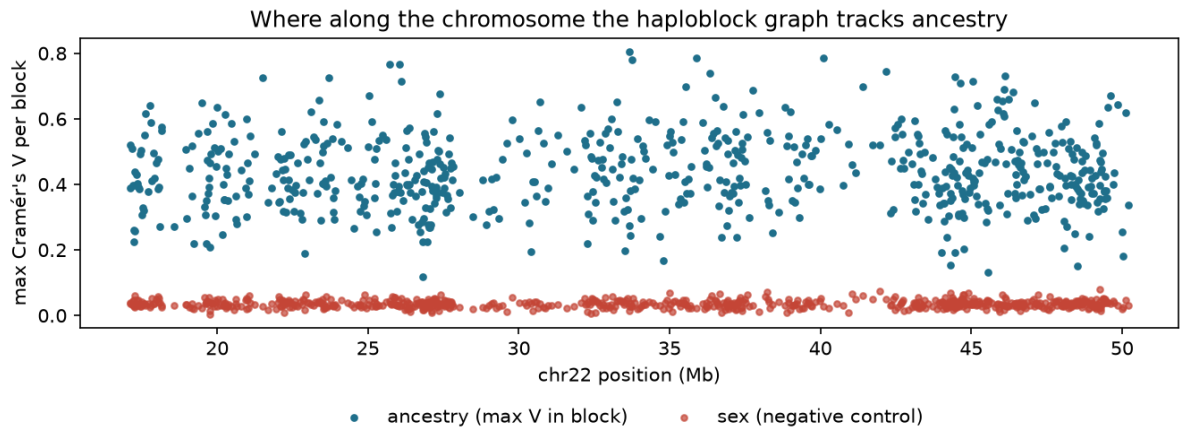


Figure 15: Maximum Cramer's V per block along chr22 for ancestry versus sex.

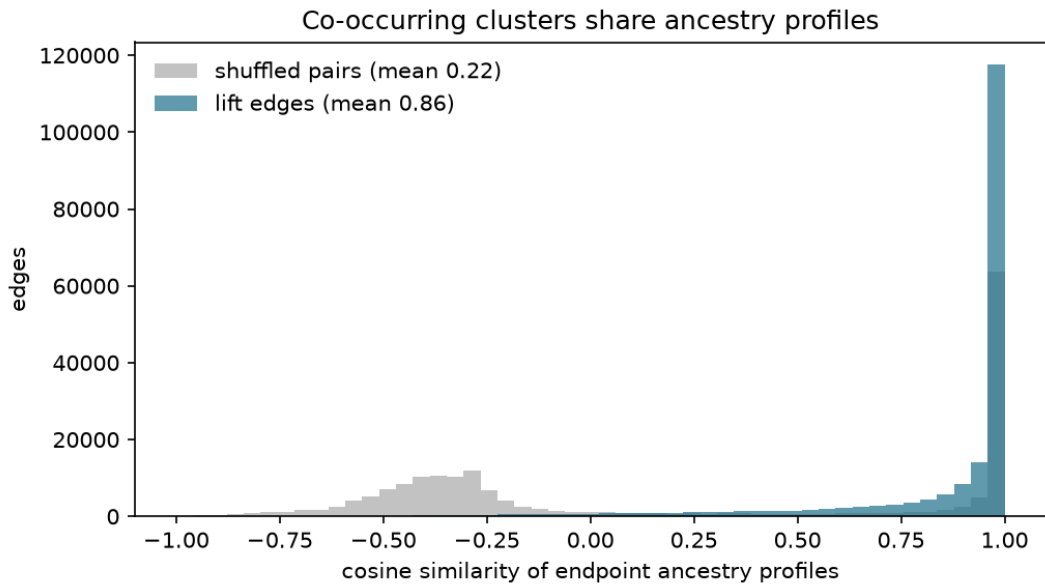


Figure 16: Co-occurring clusters share ancestry profiles: real lift edges versus shuffled pairs.

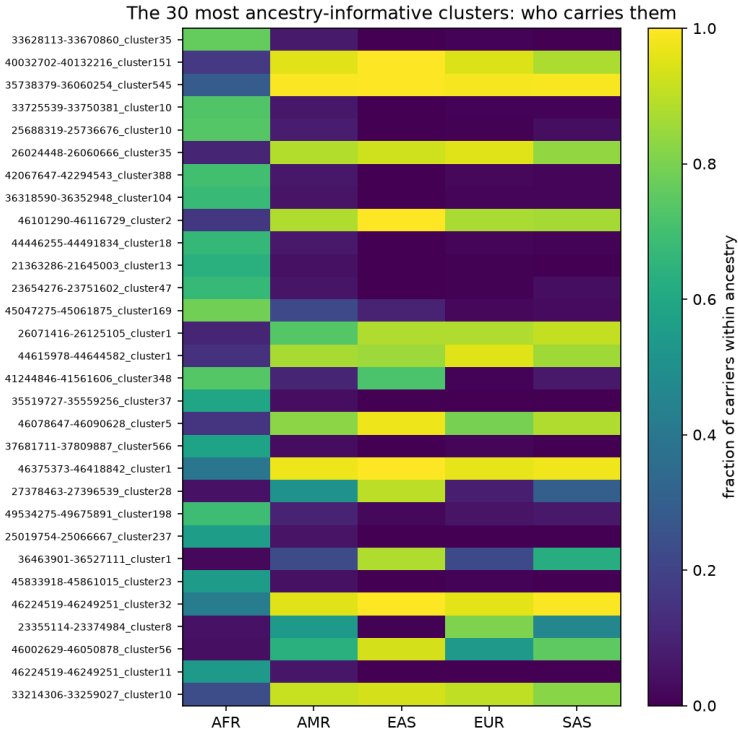


Figure 17: The 30 most ancestry-informative clusters and the fraction of each ancestry that carries them.

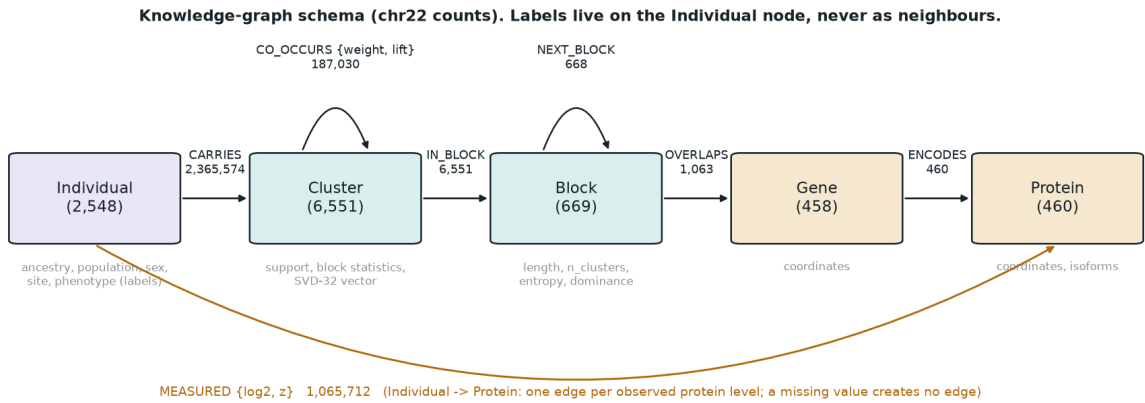


Figure 18: Node and edge types of the knowledge graph with chr22 counts.

Node type	Properties	Count	Source
Individual	ancestry, population, sex, site, phenotype, age	2,548	nodes.csv.gz columns + phenotypes + proteomics metadata
Cluster	support, block statistics, SVD vector	6,551	nodes.csv.gz rows after the filter
Block	length, n_clusters, entropy, dominance, singleton rate	669	block_stats.tsv
Gene	coordinates, number of proteins	458	uniprot_chr22.bed
Protein	coordinates, number of isoforms	460	uniprot_chr22.bed

Edge type	Meaning	Count
Individual -CARRIES→ Cluster	the person carries this haplotype cluster (the 1s of the matrix)	2,365,574
Cluster -CO_OCCURS{weight, lift}→ Cluster	the two clusters co-occur in people more than chance	187,030
Cluster -IN_BLOCK→ Block; Block -NEXT_BLOCK→ Block	position on the chromosome	6,551; 668
Block -OVERLAPS→ Gene; Gene -ENCODES→ Protein	coordinate intersection; protein product	1,063; 460
Individual -MEASURED{log2, z}→ Protein	one edge per observed protein level; missing values create no edge	1,065,712

Two design rules matter. First, phenotypes are properties of the person node, never nodes or edges: if a Phenotype node were connected to the person, a two-layer GNN would read the label from its neighbour and report a meaningless 100%. Second, because people are their own node type, the site boundary is a clean cut: the cluster, block, gene and protein graph is public and identical at every site, while a site holds only its people and their CARRIES and MEASURED edges.

The graph is stored as a PyTorch Geometric HeteroData object (hetero.pt, hetero_v2.pt) for modelling and loaded into Neo4j community edition (docker compose up neo4j; neo4j_load.py) for browsing at localhost:7474, where a query such as MATCH (i:Individual {id:'HG00096'})-[:CARRIES]→(c:Cluster)-[:IN_BLOCK]→(b:Block) RETURN i,c,b shows a person’s haplotypes with their blocks. NetworkX (with the nx-cugraph GPU backend in the image) produces statistics and GraphML for Gephi.

4.7 7. From graph to embeddings

A GNN needs a starting vector for every node. Cluster and block nodes use their statistics (support, block length, entropy, dominance, and so on), standardised. People and clusters together get a truncated singular value decomposition of the carrier matrix M (2,548 x 6,551): M is approximated as $U S V$ -transpose with 32 components; the rows of $U S$ are 32-number vectors for people and the rows of $V S$ are 32-number vectors for clusters, in one shared space, so a person sits near the clusters they carry and near people with similar haplotypes. No labels are used, so nothing can leak into the test set. Alternatives implemented and compared: Node2Vec random-walk embeddings on the person-cluster graph (GPU, via pyg-lib; with 50 pretraining epochs it reaches 0.950 on ancestry against 0.974 for SVD, with the same 0.70 silhouette), free learned embeddings (0.585), and the raw 6,551-long carrier row.

Where the phenotypes enter: only as training targets. The GNN is trained to predict the label from a person’s neighbourhood; the loss reshapes all weights so the learned 64-number vectors separate the phenotype groups. The quality of a space is measured by the silhouette score (how compact

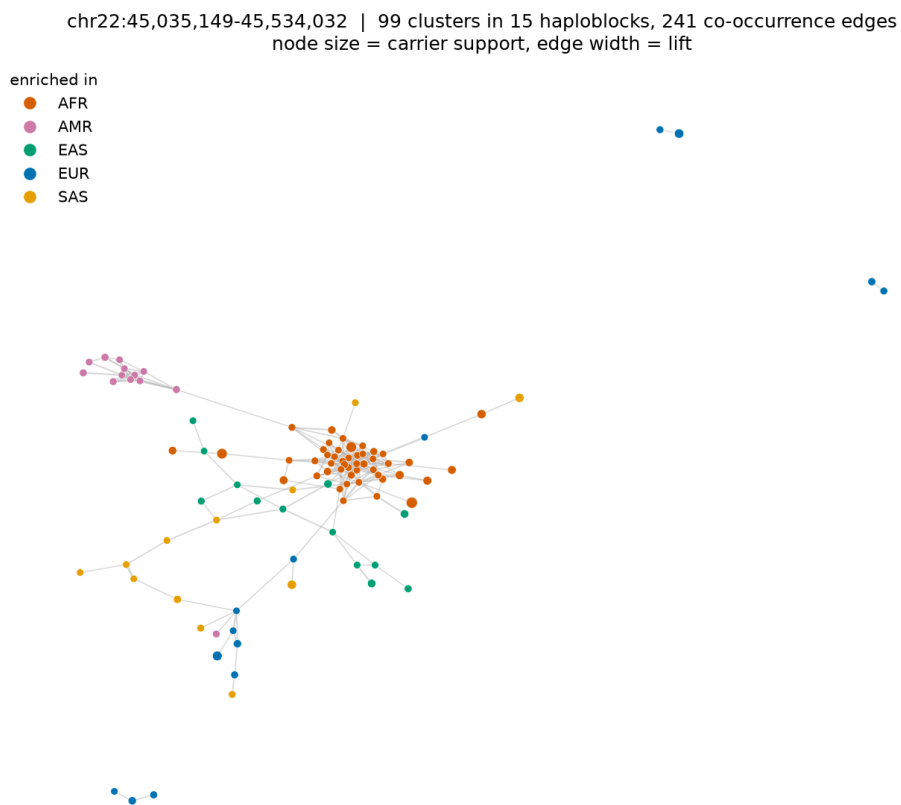


Figure 19: One region of chr22 (the densest published island): clusters as nodes coloured by the ancestry they are enriched in, co-occurrence edges weighted by lift.

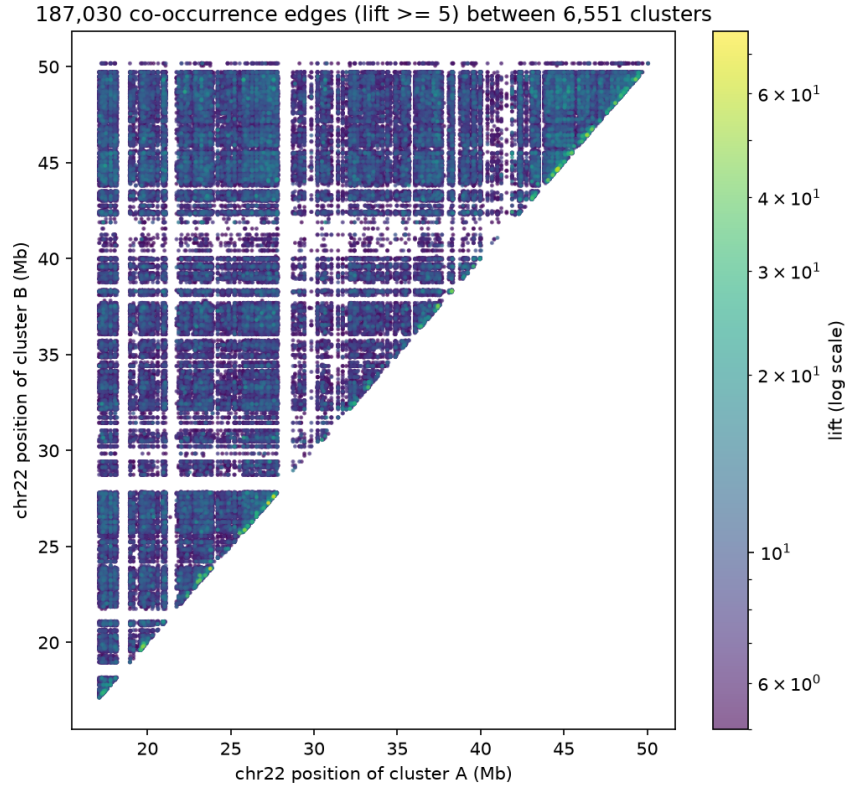


Figure 20: All 187,030 co-occurrence edges plotted by the positions of their two endpoints: block structure and long-range population structure.

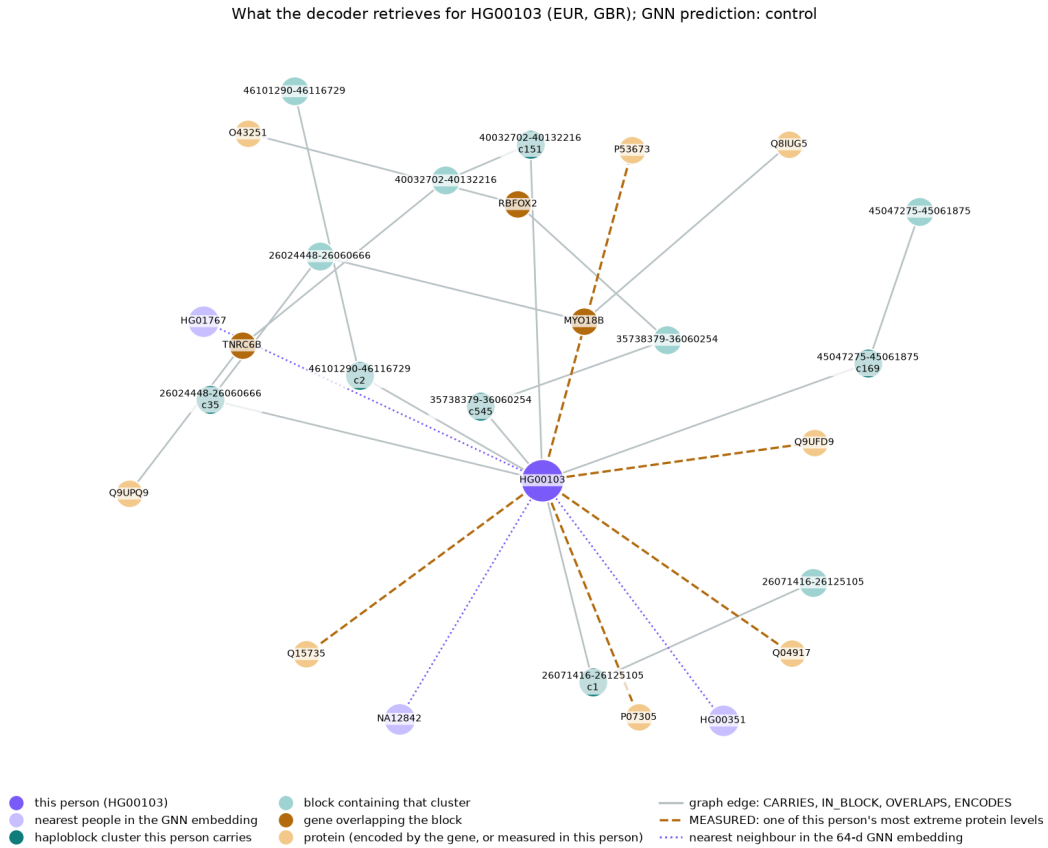


Figure 21: A real person's neighbourhood in the graph (HG00103), as the decoder retrieves it: clusters, blocks, genes, proteins, extreme protein levels and nearest neighbours in the embedding.

and separated the groups are) and by a 5-nearest-neighbour classifier: plain SVD scores silhouette 0.06 and 5-NN accuracy 0.90 by ancestry; the GNN’s hidden layer scores 0.70 and 0.97. That 0.06 to 0.70 is the GNN’s contribution. With free learned embeddings instead of SVD initialisation the GNN reaches only 0.585 balanced accuracy on ancestry: the starting embedding matters more than the architecture.

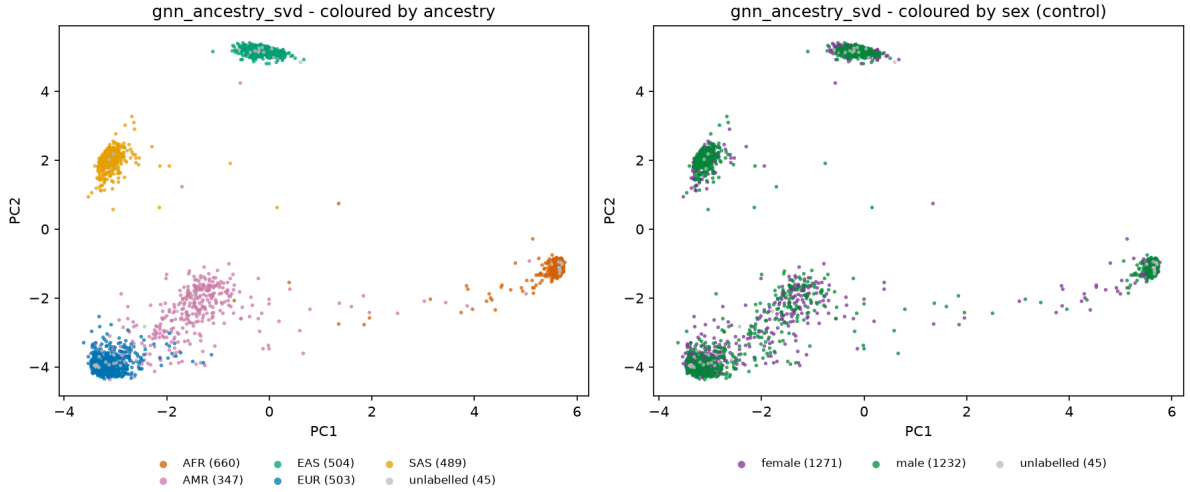


Figure 22: The GNN’s 64-dimensional embedding of people projected to two dimensions: five ancestry groups separate cleanly (AMR spread between EUR and AFR, as admixture predicts); coloured by sex the same points are fully mixed.

4.8 8. The encoder: a heterogeneous graph neural network

Architecture (PyTorch Geometric HeteroConv, two layers, hidden size 64): a linear projection per node type to 64 numbers; then two rounds of message passing in which, for every relation, SAGE-Conv adds the mean of a node’s neighbours to its own state (carries, in_block, next_block, overlaps, encodes, each in both directions) and GraphConv adds an edge-weighted mean for co_occurs (weight = normalised log lift) and measured (weight = the protein’s harmonised z-score, signed, so a high protein pushes positively and a low one negatively); after each round LayerNorm, a residual connection, ReLU and dropout 0.3; a linear head from the person’s 64 numbers to class scores. After round one a person has absorbed their clusters (and proteins); after round two, what those clusters co-occur with, their blocks, and the genes and proteins in those blocks.

Training: class-weighted cross-entropy on training people only (weights inversely proportional to class size so AMR and small populations count); Adam with learning rate 0.005 and weight decay 0.0005; early stopping on validation balanced accuracy with patience 30, best weights restored. Full-batch: one epoch is one pass over the whole graph (about 2.4 million CARRIES, 0.37 million CO_OCCURS and 1.07 million MEASURED edges), 0.95 s on an M2 laptop CPU and 0.10 s on an A100 GPU. About 105 thousand parameters with SVD input, about 500 thousand with the raw carrier row.

Target (real labels)	Classes	Logistic regression	GNN
ancestry	5	0.977	0.974 (SVD input)
population	26	0.614	0.611 (raw input; 0.437 with SVD-32)
sex (negative control)	2	0.463	0.503

Reading: balanced accuracy on the held-out 376 people. On chr22 alone ancestry and population

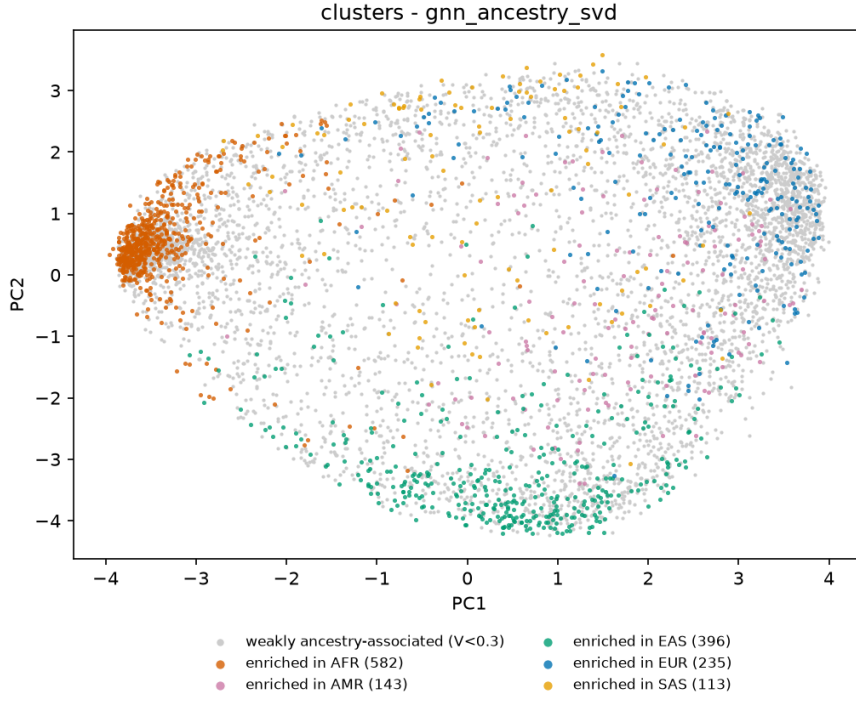


Figure 23: The same model’s embedding of clusters, coloured by the ancestry each cluster is enriched in.

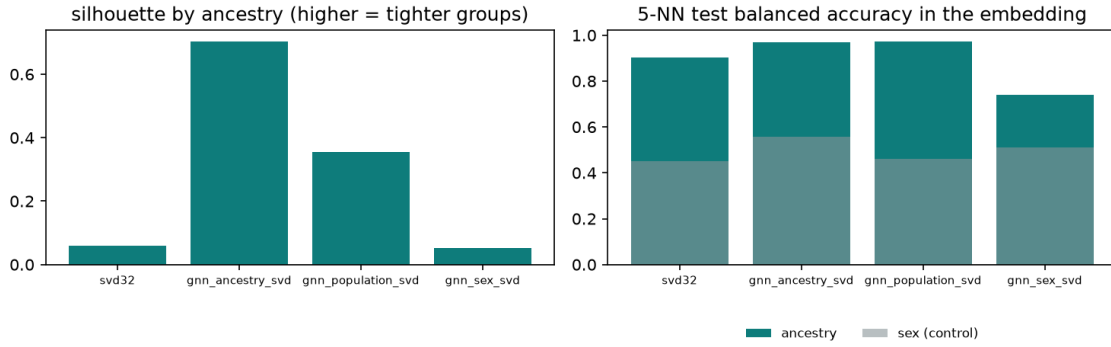


Figure 24: Embedding quality: silhouette by ancestry and 5-nearest-neighbour accuracy for SVD and for each GNN’s hidden layer; sex stays at chance.

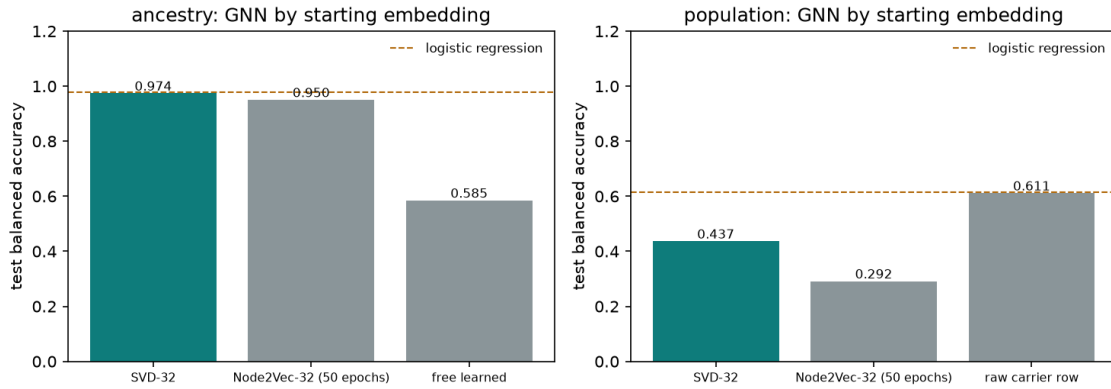


Figure 25: The same GNN with different starting embeddings for the person nodes: SVD-32, Node2Vec-32 (50 pretraining epochs, A100), free learned embeddings and the raw carrier row, against the logistic-regression line.

are almost linear functions of which clusters a person carries, so the GNN ties the linear baseline on accuracy; it wins on the embedding space and, as the next section shows, on integration.

4.8.1 Metrics, defined

- Accuracy: correct predictions divided by all predictions. Misleading when classes are unequal (predicting control for everyone scores 64% on the phenotype).
- Balanced accuracy: the mean over classes of the recall of that class (correct in class k divided by the number truly in class k). Chance is $1/k$: 0.20 for ancestry, 0.038 for population, 0.5 for sex, 0.33 for site.
- Macro-F1: for each class the harmonic mean of precision and recall, averaged over classes.
- ROC-AUC: the probability that a randomly chosen case receives a higher case score than a randomly chosen control; 0.5 is chance, 1.0 is a perfect ranking. Only defined for two classes.
- R-squared per protein: 1 minus the residual sum of squares divided by the total sum of squares, on test people with an observed value (at least five); negative means worse than predicting the mean.
- Silhouette: for each person, the mean distance to their own group minus the mean distance to the nearest other group, scaled to -1..1; averaged. Higher means tighter, better separated groups in the embedding.
- 5-nearest-neighbour accuracy: label a test person by majority vote of the five nearest training people in the embedding; balanced accuracy of that vote.
- Precision at 20: of the twenty clusters with the highest saliency, the fraction that are planted causal clusters; chance is $20 / 6,551 \times 20 = 0.06$.
- Cramer's V: $\sqrt{\text{chi-square} / N}$ for a $2 \times k$ table; 0 means the cluster is independent of the label, 1 means it determines it.
- Early stopping and model selection use the validation people only; every number reported in this document is on the test people.

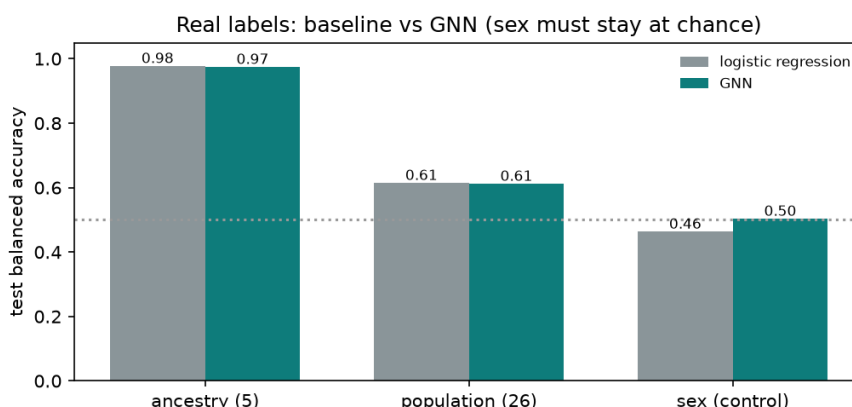


Figure 26: Real labels: logistic regression versus the GNN; sex, the negative control, stays at chance for both.

4.9 9. Integrating the proteome (schema v2)

The synthetic proteome we generated is keyed to the real 1000G ids and has a saved ground truth (ground_truth.json). Per person: a site assigned at random within each ancestry (so site is a pure batch effect), the real sex, a random age; a case/control phenotype whose log-odds is a weighted sum over 20 causal haploblock clusters plus a small age term, calibrated to 38% cases; each causal cluster also shifts one protein encoded in its own block (a cis effect); 15% of proteins respond to the phenotype, all respond to age and sex; a per-site batch shift; and missingness that increases toward the detection limit. The first version shifted every protein with the phenotype and every model scored 1.0, the same trap an earlier team result fell into, so the signal was made sparse.

Before entering the graph, protein levels pass a harmoniser: within each site and protein, $z = (\text{value} - \text{median}) / (1.4826 \times \text{median absolute deviation})$. Computed from each site's own samples, it removes the between-site shift completely (0.48 to 0.00 log2) while 227 proteins keep their phenotype association. Missing values stay missing; they simply create no MEASURED edge and are never imputed as zero.

Model	Input to the person node	Graph relations	AUC	Balanced accuracy
genome only	SVD-32 or raw carrier row	genome relations	0.60-0.63	0.57-0.62
proteome only (MLP, no graph)	harmonised z + observed mask (920 numbers)	none	0.96	0.96
genome + proteome (graph)	both	all twelve relations	0.99	0.97
site (batch control, full graph)	both	all	-	0.30 (chance 0.33)
ancestry (full graph)	both	all	-	0.90

The graph adds signal on top of the proteome, and the site control shows none of it is batch. A gradient saliency on the raw carrier input ranks clusters by their influence on the case score: 3-4 of the top 20 are ground-truth causal clusters (1.2 expected by chance). One negative result is kept deliberately: predicting the proteome from the genome embedding gives R-squared near zero even for cis-affected proteins, whereas a per-protein ridge regression on the carrier row recovers the strong cis effects (4 of 20 cis proteins with test R-squared above 0.1, 0 of 440 others). The GNN is the integration and embedding tool; discovering single-cluster cis effects needs sparse per-protein models or a prior from published protein-QTL data.

4.10 10. The decoder: GraphRAG with an NVIDIA NIM language model

For one person the decoder retrieves, deterministically and only from the graph: the profile (ancestry, population, sex, site, age; the true phenotype withheld), the GNN's prediction, the globally salient clusters the person carries, their eight most ancestry-informative clusters with block, genes and proteins, their eight most extreme protein levels with the encoding block and whether that block holds a notable cluster, and their five nearest people in the GNN embedding. This is serialised as JSON (about 4,500 tokens) and sent to nvidia/nemotron-3-super-120b-a12b through NVIDIA's OpenAI-compatible NIM endpoint with reasoning_effort set to none (otherwise this reasoning model thinks inline and exhausts the token budget before answering) under a system prompt that forbids inventing entities and requires every id to be cited verbatim. The reply's cited ids are checked against the context before anything is written. For person HG00103 the model answered in 13 seconds with 25 cited ids and none unknown, producing a summary, ancestry and phenotype assessments, genome-proteome links (for example cluster chr22_40032702-40132216_cluster151 in the block encoding

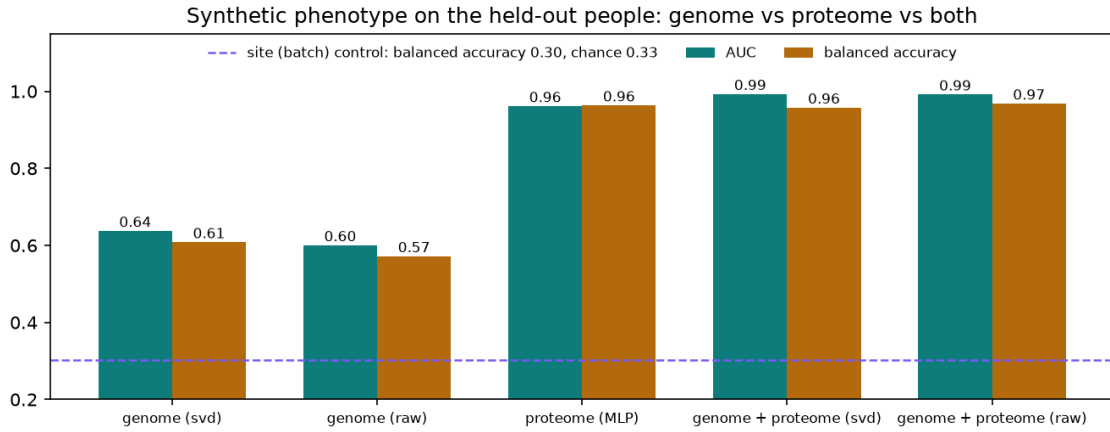


Figure 27: Held-out AUC and balanced accuracy for the synthetic phenotype by modality; the site control sits at chance.

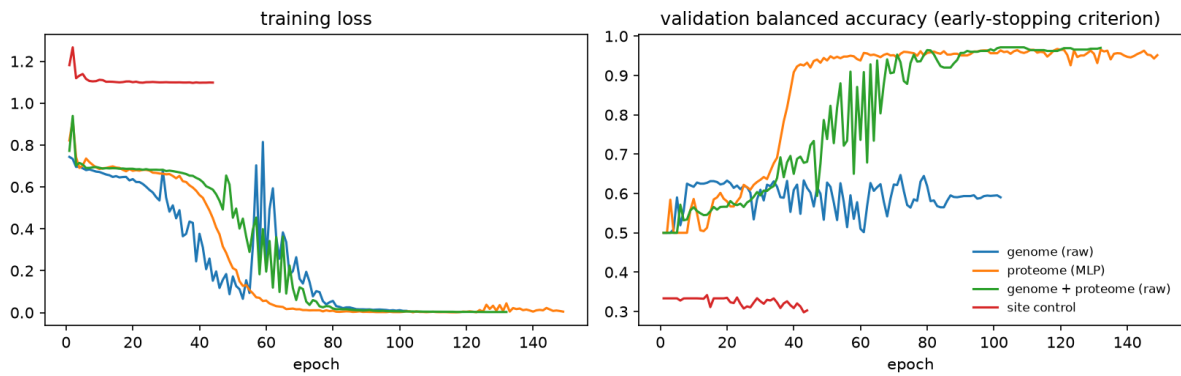


Figure 28: Training loss and validation balanced accuracy per epoch for the v2 runs (early stopping picks the best validation epoch).

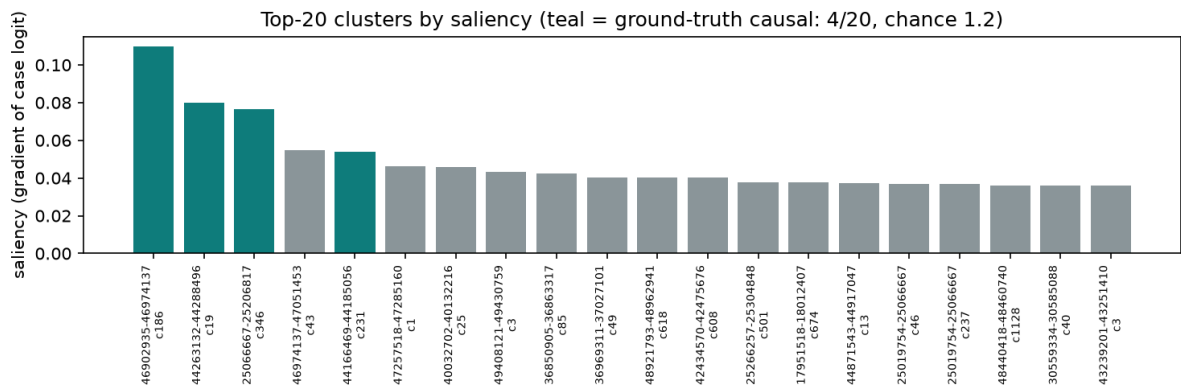


Figure 29: Cluster saliency of the combined model: ground-truth causal clusters (teal) among the top 20.

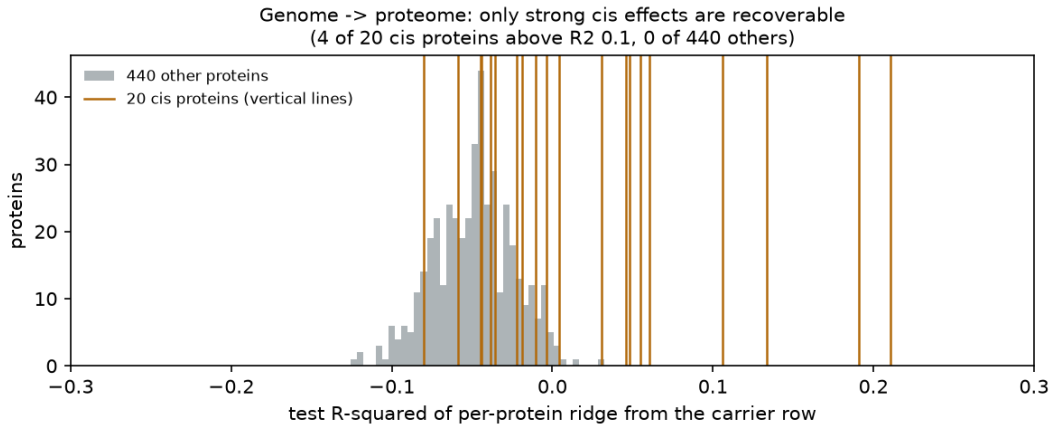


Figure 30: Per-protein ridge from the genome: only the strong cis effects are recoverable.

TNRC6B / Q9UPQ9) and caveats stating that the phenotype is synthetic and ancestry is population structure, not a medical finding.

4.11 11. Federated learning with NVFlare

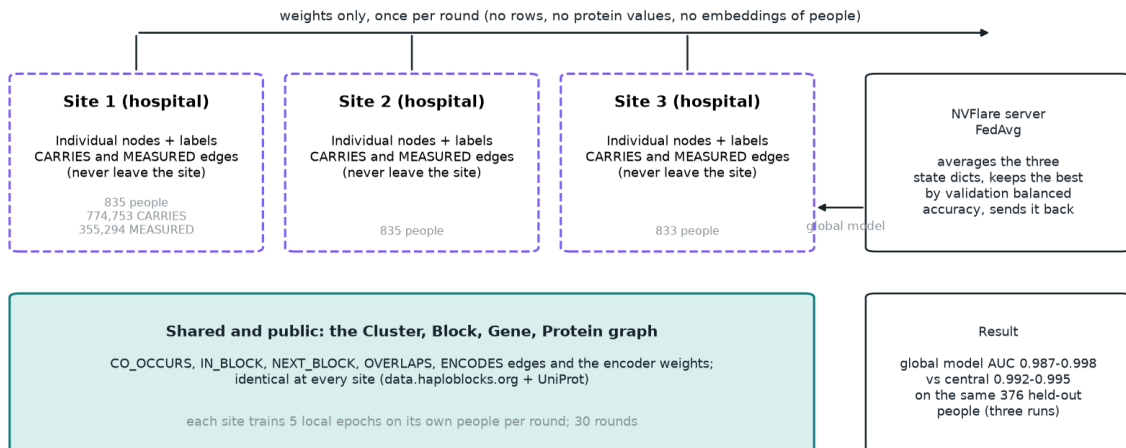


Figure 31: Federated topology: three sites with private people and edges, one shared public graph, weights only to the server.

Partition: site s receives the people whose site code is s (835 / 835 / 833, mixed ancestry). Its graph is PyG's `HeteroData.subgraph` restricted to those people: their nodes are re-indexed, their CARRIES and MEASURED edges kept, and every other node type and edge kept whole because those are public. Site 1, for instance, holds 835 people, 774,753 CARRIES and 355,294 MEASURED edges. The person's input (carrier row plus harmonised protein vector and mask) is computed from the site's own rows; no cross-site preprocessing exists because the harmoniser is already per site.

Mechanics (NVFlare 2.9, FedAvgRecipe with the PyTorch Client API): `job.py` derives every model constructor value from the public graph into `model_args.json` so the server and all clients build byte-identical models. Each client runs `flare.init()`, builds its site graph once, then loops: receive the global weights, evaluate them on its own validation and test people, train five full-batch epochs on its own training people, send back the weights, the metrics and the number of optimizer steps. The server averages the weights (weighted by steps, equal here) and selects the best global model by validation balanced accuracy. What crosses the site boundary per round: one state dict of

about half a million numbers and five scalars; no rows, no protein values, no embeddings of people.

Model	Balanced accu- racy	AUC	Evaluated on
federated global model, 10 rounds x 5 epochs	0.90	0.955	the same 376 held-out people
federated global model, 30 rounds x 5 epochs	0.963	0.998	the same 376 held-out people
central model (train_gnn_v2, both/raw)	0.969	0.992	the same 376 held-out people
federated model per site (30 rounds)	0.983 / 0.933	0.981 / 0.997	1.000 / 0.999 / 0.997 each site's own held-out people

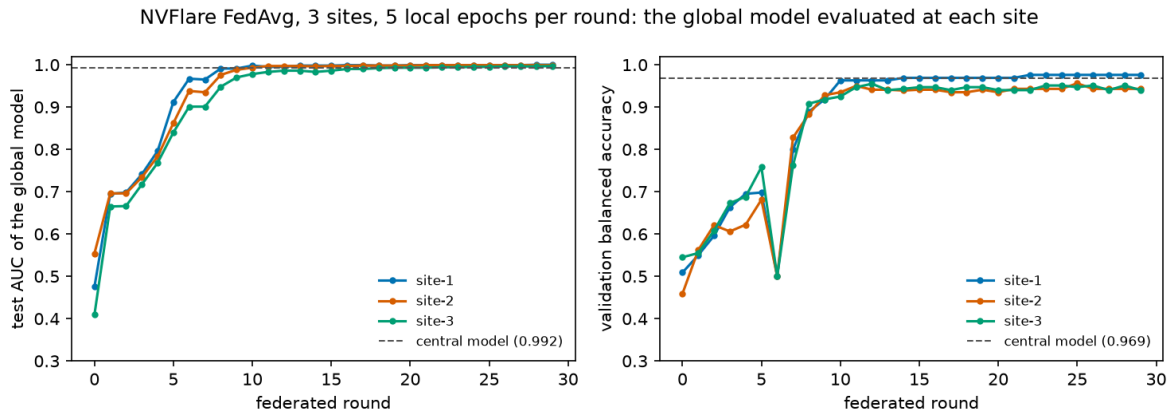


Figure 32: FedAvg convergence: the global model’s AUC and validation balanced accuracy at each site, per round, against the central model.

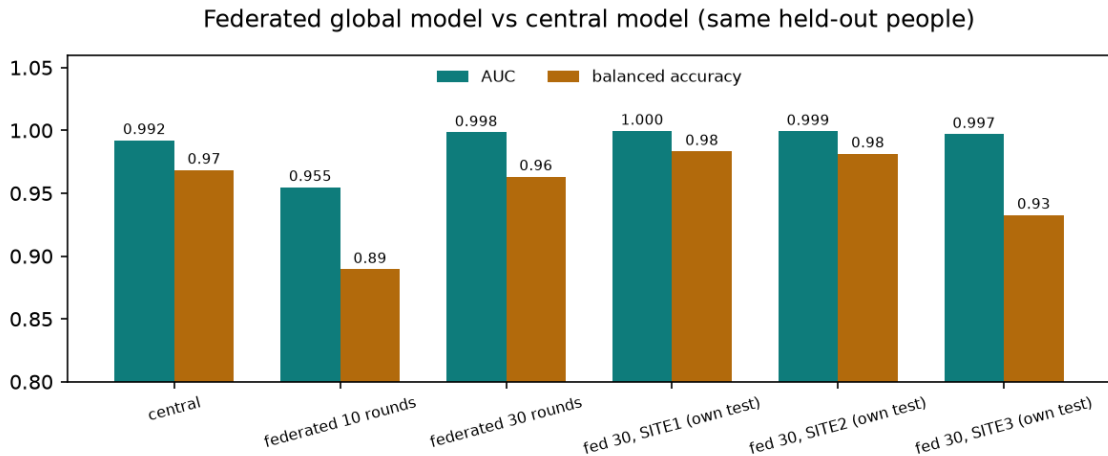


Figure 33: Central versus federated (10 and 30 rounds) on the same held-out people, and the federated model on each site’s own held-out people.

Ten rounds (50 local steps) were short of the central run’s ~80 epochs; thirty rounds converge to the central model’s level. Federated training loses nothing here because the sites are random draws of the same population by construction; with ancestry-pure sites the averaging would have to fight client drift, which is the next experiment.

4.11.1 With and without federation, on this data

federated/local_only.py trains each site alone on its own people for the same 150 optimizer steps the federated clients used, then scores that lone model on its own held-out people and on the other sites' held-out people. On this synthetic data a lone site already does well, because 835 people and a strong proteome signal are enough: own-site AUC SITE1 0.998, SITE2 0.986, SITE3 0.988; the worst transfer of a lone model to another site's people is SITE1 0.949, SITE2 0.969, SITE3 0.997. The federated global model scores SITE1 0.999, SITE2 1.000, SITE3 0.968 on the same per-site people and 0.987 to 0.998 on the pooled held-out set across runs. With a smaller budget the picture changes: at 50 optimizer steps per site (a 10-round run inside the Docker image on the A100) a lone site reaches only 0.88 to 0.92 while the federated model reaches 0.989 to 0.998 on the same per-site people, because the averaged weights have effectively seen every site's people. So federation costs nothing when a site has enough data and budget and helps clearly when it does not; in both cases what it solves is the constraint, not the score: one shared model, trained on everyone, with no row leaving any site. A larger gain is expected when sites differ systematically (ancestry-pure hospitals, different protein panels), which is the ancestry-partitioned experiment listed under next steps.

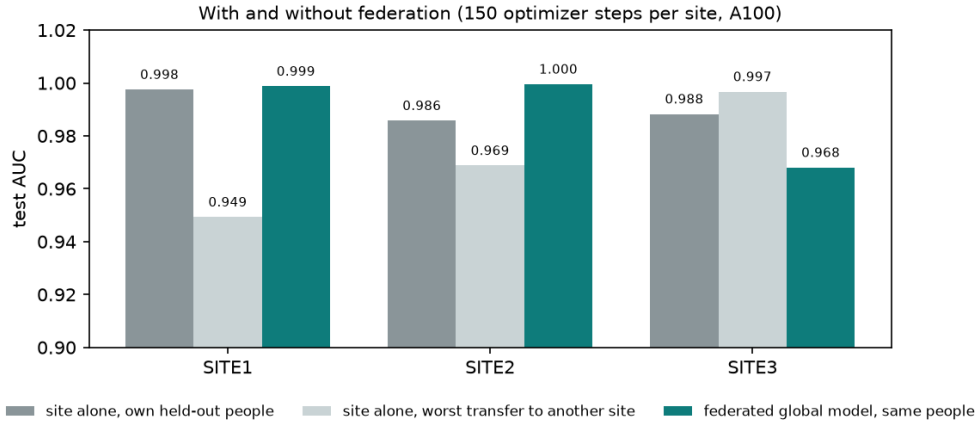


Figure 34: With and without federation: each site trained alone on its own people versus the federated global model, scored on the same held-out people per site (A100 re-run).

Site (held-out n)	Alone, 150 steps	Federated, 150 steps	Alone, 50 steps	Federated, 50 steps
SITE1 (111)	0.998 / 0.952	0.999 / 0.983	0.881 / 0.543	0.998 / 0.950
SITE2 (136)	0.986 / 0.852	1.000 / 0.991	0.902 / 0.772	0.991 / 0.907
SITE3 (129)	0.988 / 0.923	0.968 / 0.933	0.915 / 0.830	0.989 / 0.865
average over sites	0.991 / 0.909	0.989 / 0.969	0.899 / 0.715	0.993 / 0.907
pooled 376 people, federated global model	-	0.987 / 0.967	-	0.991 / 0.901
pooled 376 people, central model	0.995 / 0.960		0.997 / 0.959	

AUC / balanced accuracy on each site's own held-out people. Transfer of a lone model to other sites' people (150 steps, AUC): the SITE1 model scores 0.982 on SITE2 people and 0.949 on SITE3 people; SITE2: 0.993 and 0.969; SITE3: 0.997 and 0.998; the federated model 0.999 / 1.000 / 0.968 on the three sites.

4.11.2 Why federated, and what kind

The mission sentence of the README is a statement about where data lives: a hospital may compute on its patients but may not ship their rows. This is horizontal federated learning: every site has the same columns (the same graph schema, the same protein panel) and different rows (different people). It is not a split of the genome across sites (each site would then hold part of every person, which is vertical federation and a different problem) and not a way to add chromosomes: another chromosome is another shared reference graph, added at every site at once. What is gained is the ability to train on all 2,503 people while each site only ever reads its 835; what is lost, in this experiment, is nothing measurable, because the sites are alike. A real deployment would report each site's own held-out score and never assemble a central test set.

4.11.3 Mechanics in detail

- Model definition (federated/model.py): the same architecture as `train_gnn_v2`, copied into a self-contained file so the NVFlare server can import it without the pipeline. Its constructor reads `model_args.json` (input sizes per node type, the twelve relations, hidden 64, two layers, dropout 0.3, mean aggregation) so server and clients build identical state dicts.
- Job (federated/job.py): `FedAvgRecipe(name, model class path and args, min_clients 3, num_rounds, train_script client.py, train_args, key_metric val_balanced_accuracy, server_expected_format PYTORCH)`; `add_decomposers` registers `TensorDecomposer` so tensors travel natively; `add_server_file` ships `model.py` to the server; `SimEnv(num_clients 3, workspace_root)` runs the three sites as threads on one machine; `recipe.execute(env)` writes the job and runs it.
- Client (federated/client.py): `flare.init()`; the site name (site-1, site-2, site-3) selects the site code; `HeteroData.subgraph` keeps that site's people and their CARRIES and MEASURED edges and leaves the public node types whole; then the loop: `flare.receive()` gives the global weights and the round number; evaluate them on the site's validation and test people; if the task is evaluate-only, send metrics; otherwise train five full-batch epochs (five optimizer steps) with the site's own class weights and send `FLModel(params = state dict on CPU, metrics, meta NUM_STEPS_CURRENT_ROUND = 5)`.
- Server: after each round the global weights become the step-weighted mean of the three state dicts (weights equal here because every site does five steps; `aggregation_weights` can weight by site size); the best global model by validation balanced accuracy is kept as `best_FL_global_model.pt`, the final one as `FL_global_model.pt`, both under the workspace's `app_server` folder.
- What crosses the boundary per round per site: one state dict of about 0.5 million floats and five scalars. No carrier row, no protein value, no person embedding. Weight updates can in principle leak information about training data; NVFlare offers differential privacy and homomorphic-encryption filters for that, and neither was needed for a simulation.
- Evaluation (federated/evaluate_global.py): loads `FL_global_model.pt`, rebuilds the full graph exactly as the central run did, scores the same 376 held-out people and each site's own held-out people, and writes `evaluation.json` next to the central numbers.
- From simulation to real machines: NVFlare's POC mode starts a server and clients as separate processes (or machines) with the same job; each client would run `client.py` against its own `kg-dir` and split; the L4 and A100 instances could be two such sites. Adding a real fourth site means: its own people with 1000G-style ids, its own protein matrix keyed by those ids, its own harmoniser pass, and the shared graph files copied over.

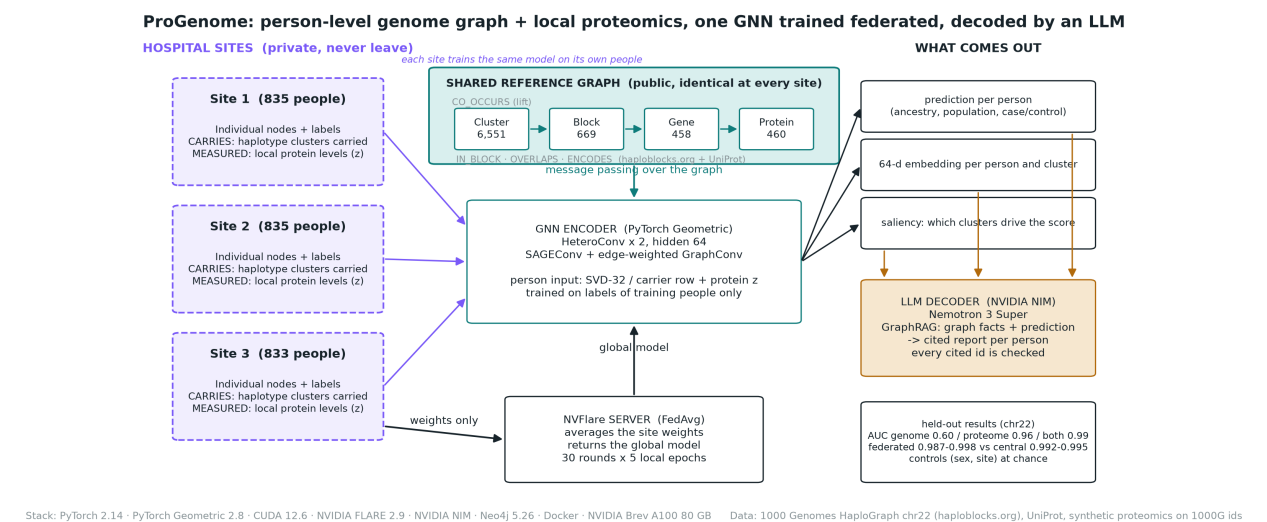


Figure 35: System architecture on one page: private hospital sites, the shared reference graph, the GNN encoder trained through the NVFlare server, and the outputs feeding the LLM decoder.

4.12 12. System design, tech stack and deployment

Layer	Choice	Notes
Language and data	Python 3.13, pandas 3.0, scipy 1.18 (sparse), scikit-learn 1.9	pinned in requirements.txt
Graph learning	PyTorch 2.14, PyTorch Geometric 2.8, pyg-lib (random walks for Node2Vec)	torch_cluster is deprecated in favour of pyg-lib; that caused the first image-build failure
Graph tooling	NetworkX 3.6, nx-cugraph (GPU dispatch), Neo4j 5.26 community in Docker	graph statistics, GraphML, browsing
Container	pytorch/pytorch:2.14.0-cuda12.6-cudnn9-runtime base, PIP_BREAK_SYSTEM_PACKAGES=1, torch-tensorrt 2.14	10.5 GB image; runs on CPU when no GPU is present
Compute	Mac M2 CPU for development; NVIDIA Brev: L4 24 GB (GCP, \$0.85/h) and A100 80 GB (Crusoe, \$1.98/h)	brev_deploy.sh creates or reuses an instance, uploads, builds natively, runs, copies outputs back
Inference	torch.compile (inductor): 36 ms → 4.9 ms per full graph, logits equal to within 0.0001	Torch-TensorRT could not compile this scatter-heavy GNN within 3 hours and is not claimed key in the environment only; a local NIM container on the A100 would keep patient context on site
LLM	NVIDIA NIM, nvidia/nemotron-3-super-120b-a12b	
Federated	NVFlare 2.9 FedAvgRecipe, SimEnv simulator; federated/local_only.py for the site-alone comparison	3 clients as threads on one machine; POC mode across real machines is the next step
Packaging	Makefile (setup, run, run-v2, eda, decode, federated, docker, docker-run-v2, docker-federated, brev, report), setup.sh, run_all.sh, run_v2.sh, config.py + .env.example, 11 unit tests	clone-and-run on laptop or GPU; the image is the whole solution (data mounted, .env passed)

Runtime, measured on 18 September from an empty folder (the verification runs of section 13):

Stage	Laptop, Apple M2 CPU	A100 80 GB (Docker)
environment: venv or image, pinned dependencies, unit tests	28 s (uv)	3 min image build
v1: download, knowledge graph, statistics, baseline, plots, three GNN runs, embedding quality	6 min	3 min
v2: synthetic proteomics, graph v2, EDA, six GNN runs, ridge, decoder	19 min	4 min
federated: 30 rounds x 5 local epochs, central scoring, site-alone comparison	4 min	2 min
whole chain	about 30 min	about 14 min including the image build
one GNN training run	30 s to 3 min	1 to 16 s
one full-graph inference pass (2,548 people)	-	36 ms eager, 4.9 ms compiled
one decoder call (NIM, remote)	4 to 13 s	same

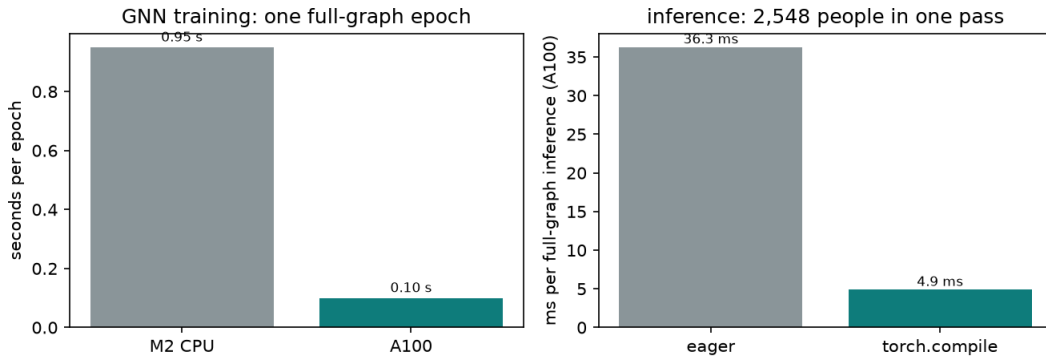


Figure 36: Training epoch time on the laptop CPU versus the A100, and full-graph inference eager versus torch.compile on the A100.

4.12.1 The Docker image, layer by layer

The Dockerfile starts from `pytorch/pytorch:2.14.0-cuda12.6-cudnn9-runtime` (PyTorch with CUDA 12.6 already inside), sets `PIP_BREAK_SYSTEM_PACKAGES=1` because the base image’s Python is system-managed, installs `curl`, copies `requirements.txt` and installs the pinned libraries, then tries three optional extras and prints a clear fallback message if any is unavailable for this torch build: `pyg_lib` from the PyG wheel index (Node2Vec random walks; without it the code falls back to SVD), `nx-cugraph` from NVIDIA’s index (NetworkX dispatches to cuGraph on the GPU; without it NetworkX runs on the CPU), and `torch-tensorrt` from the PyTorch cu126 index. It copies the code, runs the unit tests as part of the build so a broken image cannot be produced, and defaults to running `run_all.sh`. Data and outputs are bind-mounted at run time, so the image never contains data. `docker-compose.yml` adds Neo4j 5.26 community with a persistent volume and the same pipeline image with GPU reservation.

4.12.2 GPU deployment on Brev, step by step

brev_deploy.sh needs a logged-in brev CLI (brev login -api-key). It creates the instance if it does not exist (default an L4 on GCP; BREV_INSTANCE=progenome-a100 selects the A100), waits until the instance reports RUNNING and READY, refreshes the SSH alias, checks nvidia-smi and docker over ssh, uploads genomics/ plus the two small proteomics inputs as one tarball, builds the image natively on the GPU box (no emulation, about 4 minutes on the A100), runs the whole v1 pipeline inside the container with the GPU, runs the inference benchmark (eager, torch.compile, TensorRT attempt) and copies outputs/ back to outputs_brev/<instance>/. Billing continues until brev stop; the A100 is left running for the presentation.

4.12.3 Inference and what torch.compile does

infer.py rebuilds the exact inputs of a trained run, wraps the model so torch.compile sees plain tensors instead of dictionaries, times ten full-graph passes eager, then compiles with the inductor backend (kernel fusion and graph capture) or with the torch_tensorrt backend and times again, and reports the maximum absolute logit difference so a speed-up cannot hide a numerical change. On the A100: 36.3 ms eager, 4.9 ms compiled (7.4x), maximum logit difference below 0.0001 across three runs, accuracy on all 2,503 labelled people 0.980 with SVD input. The TensorRT backend partitions the graph into dense parts it can compile (linear layers, norms) and scatter parts it cannot; on this hetero-GNN the compile step did not finish in three hours and is not part of any claim.

4.13 13. How the goals were met, and what is not claimed

Goal	Evidence	Status
RQ1: one graph joining haploblocks, genes, proteins and people	schema v2 with 5 node types and 7 edge types, 2,548 people joined by id, browsable in Neo4j	done
RQ2: GNN combines genome and proteome	AUC 0.60 (genome) / 0.96 (proteome) / 0.99 (both); site control at chance; embedding silhouette 0.70; saliency finds causal clusters at 3x chance	done on synthetic ground truth
RQ3: federated training without moving records	NVFlare FedAvg over 3 sites, AUC 0.998 vs central 0.992 on the same people	done in simulation
Encoder → LLM decoder → insights	GraphRAG decoder with validated citations on Nemotron 3 Super	done
Reproducible and deployable	Makefile, Docker image, Brev deployment on L4 and A100, tests	done

- The case/control phenotype is synthetic: the integration numbers show the pipeline recovers a planted signal, not biology. Ancestry, population and sex results are on real labels.
- Chromosome 22 only; the code is chromosome-agnostic (CHROM=chr21 make run).
- On single-label accuracy the GNN ties, not beats, logistic regression; its value is the embedding space and the integration.
- The GNN embedding does not recover single-cluster cis effects; a per-protein ridge does for the strong ones.
- Federated evaluation reuses the central held-out people; a real deployment would report per site only.

- TensorRT is not part of the inference claim; torch.compile is.

4.13.1 Verification and reproduction (18 September, before the commit)

Before committing, the whole chain was re-run twice from the files that would be committed: once as a simulated fresh clone on the laptop (only the tracked files copied to an empty folder, then make setup, make run, make run-v2, make federated, make test) and once on the A100 with the Docker image rebuilt from the current Dockerfile, which had not been built since the pyg_lib change. Both runs reproduce every count exactly and every model score within run-to-run noise (GPU kernels and FedAvg are not bit-reproducible). The rebuilt image contains pyg_lib, so Node2Vec ran natively for the first time; with its original five pretraining epochs it was clearly undertrained (ancestry 0.80), with fifty epochs the loss converges and it becomes a working but slightly weaker alternative to SVD, so SVD remains the default everywhere and the deploy script now defaults to it. The decoder was also re-called on the laptop run; the language model’s wording and the number of ids it chooses to cite vary between calls, the citation check is what stays constant.

Quantity	Documented (original runs)	Fresh clone, lap- top CPU	Rebuilt image, A100
kept clusters / CARRIES / CO_OCCURS	6,551 / 2,365,574 / 187,030	identical	identical
genes / proteins / MEASURED edges	458 / 460 / 1,065,712	identical	identical
ancestry-associated clusters (FDR 5%) / sex	6,470 / 0	6,470 / 0	6,470 / 0
logistic baseline ancestry / population / sex	0.977 / 0.614 / 0.463	0.977 / 0.614 / 0.463	0.981 / 0.614 / 0.463
GNN v1 ancestry / population / sex (SVD input)	0.974 / 0.437 / 0.503	0.974 / 0.437 / 0.503	run with Node2Vec instead (next row)
GNN v1 with Node2Vec input, 50 epochs (A100 only)	not previously measured	-	0.950 / 0.292 / 0.489
embedding silhouette by ancestry: SVD / GNN	0.06 / 0.70	0.06 / 0.70	0.06 / 0.70 (Node2Vec-initialised GNN)
phenotype AUC genome / proteome / both	0.64 / 0.96 / 0.99	0.65 / 0.96 / 0.99	0.64 / 0.96 / 0.99
site control balanced accuracy (chance 0.33)	0.30	0.34	0.30
saliency: causal clusters in top 20	4	4	4
ridge: cis proteins with R2 > 0.1 / others	4 / 0	4 / 0	4 / 0
federated 30 rounds AUC / central AUC, same 376 people	0.998 / 0.992	0.996 / 0.994	0.987 / 0.995
site-alone AUC on own test people vs federated model on the same people	not previously measured	-	0.998 / 0.986 / 0.988 vs 0.999 / 1.000 / 0.968
decoder: cited ids / invented ids	25 / 0	4 / 0 (4.2 s)	dry run (no key on the box)
inference eager / torch.compile (A100)	36 ms / 4.9 ms	-	36.3 ms / 4.9 ms (7.45x, max logit diff 1.9e-06)

Quantity	Documented (original runs)	Fresh clone, lap- top CPU	Rebuilt A100	image, build)
unit tests	11 pass	11 pass (make setup)	11 pass (docker build)	

4.14 14. Team context

Friederike Duendar (lead) wrote the README’s research questions, the UniProt gene BED and an R exploration of gene-block overlaps; Nolan Bruyat built the synthetic proteomics generator and its plots; Zillur Rahman built the proteomics-side analysis now on main (see below); Yan Zhou coordinates the manuscript (two introduction paragraphs, two methods paragraphs, one results paragraph); Anita Egebor and Alvaro Martinez Barrio contributed to the README and data access. The genomics/ pipeline described here is the modelling backbone into which those pieces plug.

4.14.1 What is on main since this branch was created, and how it relates

The team’s proteomics work on main (the synthetic proteomics generator with 4,000 SITE-id samples, protein filtering and classification baselines, a protein-centred knowledge graph built from the HaploGraph edge list annotated with UniProt proteins, a federated comparison of feature sets, and methods_and_results.md) covers the proteome side and the federated logistic setting. genomics/ adds the genome side: the person-level knowledge graph on the 1000 Genomes ids, the graph neural network, the LLM decoder and the NVFlare run. The two share the same HaploGraph edge file, the same UniProt gene BED and the same three-site design, and they are complementary: nothing in genomics/ touches a path on main, so the branch merges cleanly, and for the manuscript the proteomics analysis and the graph model plug into the same methods and results structure.

5 Part IV: Reference

5.1 Run it yourself

From a fresh clone of the repository on branch modelling, on a laptop or a GPU machine:

```
git clone https://github.com/collaborativebioinformatics/ProGenome.git
cd ProGenome && git checkout modelling && cd genomics
make setup          # .venv with torch (CPU, or CUDA if nvidia-smi works), pinned deps, unit tests
make run            # v1: download -> graph -> statistics -> baseline -> plots -> GNN (ancestry, popul
make run-v2         # v2: synthetic proteomics -> graph v2 -> EDA -> GNN ablations -> ridge -> decoder
make federated ROUNDS=30 LOCAL_EPOCHS=5      # NVFlare FedAvg over 3 sites, then central scoring
make neo4j-load     # browse the graph at http://localhost:7474 (neo4j / progenome)
make docker && make docker-run                 # the same v1 chain inside the CUDA image
BREV_INSTANCE=progenome-a100 make brev        # build and run on the A100, copy outputs back
make decode WHO=HG00103                      # LLM insight for one person (needs NVIDIA_API_KEY)
make test          # 11 unit tests on a toy graph
make help          # every target
```

Single stages take a `-chrom` flag and, where relevant, `-target`, `-modality`, `-init`; for example `.venv/bin/python train_gnn_v2.py -target phenotype -modality both -init raw` reproduces the combined model with saliency, and `.venv/bin/python infer.py -run ancestry_svd -compile inductor` reproduces the inference benchmark. Every output path is relative to `genomics/`, and `data/`, `outputs/` and `outputs_brev/` are git-ignored.

5.2 Secrets and configuration

Configuration enters the code in one place, `config.py`, which reads `genomics/.env`, then `~/progenome.env`, then defaults, with exported shell variables taking precedence; `.env.example` lists every variable with a comment (`NVIDIA_API_KEY`, `NIM_MODEL`, `NIM_URL`, `NEO4J_URI/USER/PASSWORD`, `HAPLOBLOCKS_BASE`, `CHROM`, data and output directories, `BREV_INSTANCE/TYPE`) and make config prints what is in effect with secrets masked. The shell scripts source the same files through `load_env.sh`, and the Docker targets pass `.env` into the container. The only secret is the NVIDIA API key; `.env` is git-ignored, nothing under the repository contains a key, and the decoder refuses to call the endpoint without one instead of falling back silently. Versions are pinned in `requirements.txt` (torch 2.14.0, torch_geometric 2.8.0.post1, pandas 3.0.5, scipy 1.18.1, scikit-learn 1.9.1, networkx 3.6.1, neo4j 6.3.1, nvflare 2.9.0, pytest 9.1.1) and `setup.sh` installs torch from the CPU or cu126 index depending on whether a GPU is present.

5.3 Repository map (genomics/)

<code>haplokg.py</code> , <code>build_kg.py</code>	knowledge graph v1 (people, clusters, blocks)
<code>haplokg_proteins.py</code> , <code>build_kg_v2.py</code>	genes, proteins, harmonised measurements (v2)
<code>proteomics_synth_1000g.py</code>	synthetic proteomics on 1000G ids with ground truth
<code>cooccurrence_analysis.py</code> , <code>eda.py</code>	statistics and the EDA report
<code>baseline.py</code>	logistic regression + the shared split
<code>graph_explore.py</code> , <code>neo4j_load.py</code>	NetworkX statistics/plots, Neo4j loader
<code>train_gnn.py</code> , <code>train_gnn_v2.py</code>	the GNN (genome; genome+proteome)
<code>embeddings.py</code>	embedding quality and plots
<code>proteome_linear_baseline.py</code>	per-protein ridge (genome -> proteome)
<code>graphrag_decoder.py</code>	NIM LLM decoder
<code>infer.py</code>	inference benchmark
<code>federated/{model,client,job,evaluate_global}.py</code>	NVFlare FedAvg
<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>Makefile</code> , <code>setup.sh</code> , <code>run_all.sh</code> , <code>run_v2.sh</code> , <code>brev_deploy.sh</code>	
<code>docs/architecture.html</code> (.mmd), <code>METHODS.md</code> , <code>DEEP_DIVE.md</code> , <code>report/</code>	
<code>tests/</code>	unit tests on a toy graph

5.4 Glossary

Term	Meaning
Haploblock	a stretch of chromosome between recombination hotspots, inherited as a unit
Haplotype	one copy's sequence of a block; each person has two per block
Cluster	a group of near-identical haplotypes of one block (MMseqs2); a person carries it if either haplotype is in it
Carrier matrix	people x clusters 0/1 matrix; the CARRIES edges
Lift	how much more often two clusters co-occur than chance: $P(A \text{ and } B) / (P(A) P(B))$
Cramer's V	effect size of a contingency test, 0 (independent) to 1 (perfectly associated)
FDR	false discovery rate; Benjamini-Hochberg controls the expected fraction of false positives among findings
SVD	singular value decomposition; here a 32-component factorisation of the carrier matrix giving embeddings
GNN / message passing	neural network on a graph; each layer mixes a node's vector with its neighbours'
SAGEConv / GraphConv	two PyG layer types: neighbour-mean aggregation; edge-weighted aggregation

Term	Meaning
Balanced accuracy / AUC	mean per-class recall; probability a random case outscores a random control
Harmoniser	per-site, per-protein robust z-score removing batch offsets
MNAR	missing not at random; here low-abundance proteins go missing first
FedAvg	federated averaging of model weights across sites
NIM	NVIDIA Inference Microservice; an OpenAI-compatible LLM endpoint
Brev	NVIDIA's GPU cloud; instances by the hour